

人文社会ビジネス科学学術院 ビジネス科学研究群 2026年度 春C

テキストマイニングの実践

講義概要

- URL: <https://github.com/haradatm/lecture-gssm2026>

注: 講義資料は、講義開始前まで更新する可能性があります。常に最新のスライドを確認するようにしてください。

①「**slides**」をクリックして開く

②「**day-N.pdf**」をクリックして開く

③ダウンロードボタン「↓」をクリック

lecture-gssm2026 / slides / day-1.pdf

10.7 MB

人文社会ビジネス科学学術院 ビジネス科学研究群 2025年度 春C
テキストマイニングの実践

演習の題材 — ホテルのクチコミ分析 (by 楽天トラベル)

- ホテルのクチコミ数: 1,635万件 ※ 2026/5/9現在

ホテルのクチコミ・お客様の声

Gemini に相談

travel.rakuten.co.jp/review/

Google で調べる

ヘルプ

ご利用ガイド

サイトマップ

Language

楽天市場

楽天グループ

Rakuten Travel

お気に入り

予約確認

ログアウト

メニュー

エントリー & 初めて使うごとに最大 6,000 ポイント GET!

原田 智彦さん 18 ポイント Bonus レベル 0

国内宿泊がポイント 1%

特別ご招待

国内宿泊

交通 + 宿

レンタカー

高速バス・ツアー

海外旅行

遊び・体験

キャンプ場

割引クーポン

ふるさと納税

旅行ガイド

その他

ホテル・旅館・高速バスのクチコミ

お客様の声

ホテルのクチコミ数 No.1 16,347,341

お客様アンケート回答数

家族で楽しめました

素敵なお部屋でした

食事が最高においしかったです!

クチコミ (お客様の声) を検索

国内宿泊 海外ホテル

検索

国内

すべてが最高

また来たい

プロポーズ

肌 つるつる

アメニティ 持ち帰り

海外

景色 最高

イケメン

日本語

新着! 最新のクチコミ

2026年5月 09日 更新

国内宿泊

海外ホテル

2026-05-08 23:59:06

ホテル富士トリエートのクチコミ (6件)

3.33

2026-05-08 23:57:49

リバーサイドホテル松栄のクチコミ (459件)

3.83

「お客様の声」には、実際にご利用になった方のご意見・ご感想が満載です。

国内宿泊

高速バス

ペットホテル

海外ホテル

スケジュール

day 1

- ・ 講義(後半) — 自然言語処理とLLM

day 2

- ・ 講義 — テキストマイニングの手順
- ・ 講義&演習 — データ理解
- ・ 講義&演習 — テキスト解析 (1)

day 3

- ・ 講義&演習 — テキスト解析 (2)
- ・ 講義&演習 — テキスト分析 (1)

day 4

- ・ 講義&演習 — テキスト分析 (2)

day 5

- ・ 演習 — テキスト分析 (発表会)
- ・ 講義&演習 — LLMを用いたテキスト分析
- ・ ラップアップ — Q&A

EXCEL

Google Colab

M365 Copilot Chat

注: 演習では、GPUランタイムは利用しません
(消費するコンピューティングユニットがより少ない
CPUランタイムを利用します)

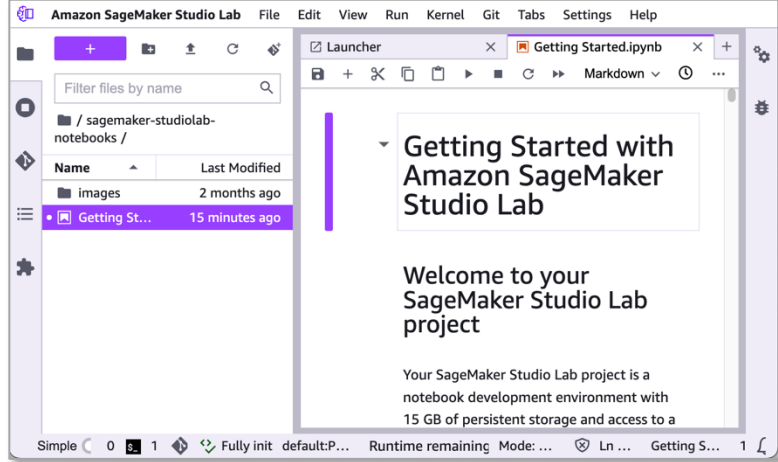
● 機械学習の教育・研究を目的とした研究用ツール



<https://colab.research.google.com>



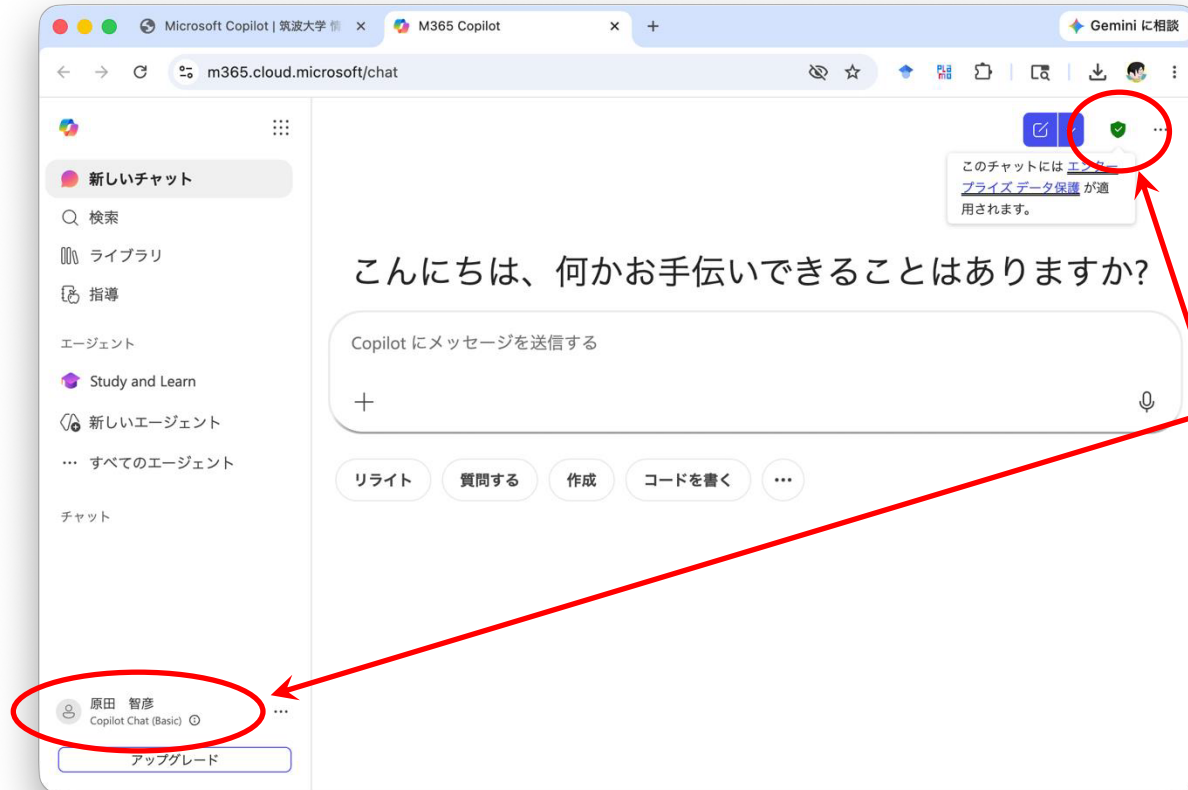
<https://studiolab.sagemaker.aws/>



演習で使用
↓

	Clab(無償版)	Studio Lab
GPU	T4(16GB)	T4(16GB)
最長 実行時間	12時間	CPU:12時間 GPU:4時間
メモリ	12GB	15GB
ディスク	CPU:100GB GPU:78GB	15GB (永続化)
ターミナル	△(限定的)	○
ランタイムの 保存と再開	×	○
費用	無償	無償
その他	Googleアカウントが必要	AWSアカウントは不要 (クレカ不要)

● Microsoft の「Microsoft 365 Copilot Chat」チャットが利用可能



[Microsoft 365 Copilot Chat](#)

利用対象者

@u または @un の本学のMicrosoft アカウントを持っている方
(離職・卒業・修了すると利用対象者ではなくなります)

利用手順

1. <https://m365.cloud.microsoft/> にアクセスします
2. 本学の @u または @un の Microsoft アカウントにてサインインしてください。
3. (既に、Microsoft 365 などにサインインしている場合でも、あらためてログイン操作が必要です)
4. Microsoft Copilot の画面が開きます。

右上に「**保護済み**」アイコンが表示され、カーソルをあわせると「このチャットにはエンタープライズデータ保護が適用されます」と表示されれば、大学の契約の下での「エンタープライズ データ保護を備えた Copilot チャット」を利用している状態です。

人文社会ビジネス科学学術院 ビジネス科学研究群 2026年度 春C

テキストマイニングの実践 day 1 (後半)

スケジュール

day 1

- 講義(後半) — 自然言語処理とLLM

day 2

- 講義 — テキストマイニングの手順
- 講義&演習 — データ理解
- 講義&演習 — テキスト解析 (1)

day 3

- 講義&演習 — テキスト解析 (2)
- 講義&演習 — テキスト分析 (1)

day 4

- 講義&演習 — テキスト分析 (2)

day 5

- 演習 — テキスト分析 (発表会)
- 講義&演習 — LLMを用いたテキスト分析
- ラップアップ — Q&A

自然言語処理と大規模言語モデル (LLM)

目次

- 大規模言語モデル(LLM)はどんなもの?
- 大規模言語モデル(LLM)の成り立ちと進化
- テキストマイニングの未来予想

大規模言語モデル (LLM) はどんなもの？

- 動画生成は、高精細な4K映像生成、リアルな動きの表現、高速・長時間生成が進む
 - OpenAI Sora, Google Veo 3, Kuaishou Kling, Luma Ray2 …



出典: Peng et al. (2025), "[Open-Sora 2.0: Training a Commercial-Level Video Generation Model in \\$200k](#)"

- 生成AIは、「作る」から「会話で編集する」へ

Nano Banana 2

高速生成、指示追従、画像検索
グラウンディング



Gemini Omini

画像・動画・音声・テキストを参照し、
動画を会話で段階的に編集



出典: [Nano Banana 2](#) および [Gemini Omni](#) (Google)で作成

大規模言語モデル(LLM)、使ってますか？

● OpenAI ChatGPT: 自然言語の入力で、何でもできる

同義語(表記揺れ)辞書が作れる

The screenshot shows a web browser window with the URL `chatgpt.com/c/6a2de582-8680-83e8-a805-c1ce736c8738`. The page displays a Python script titled "同義語辞書作成" (Synonym Dictionary Creation). The script defines a dictionary `synonym_groups` with two main categories: "宿泊施設" (Accommodation) and "風呂・温泉" (Bath/Hot Spring). Under "宿泊施設", it lists terms like "宿", "ホテル", "旅館", and "お宿" with their respective counts. Under "風呂・温泉", it lists terms like "風呂", "温泉", "浴場", "露天風呂", "湯", "お湯", "浴槽", and "洗い場" with their counts. A red callout box with a white border contains the text: "昔はこれだけで、自然言語処理の一研究テーマだった" (In the past, this alone was a research theme for one study in natural language processing).

エグサマ(分析結果のまとめ)が作れる

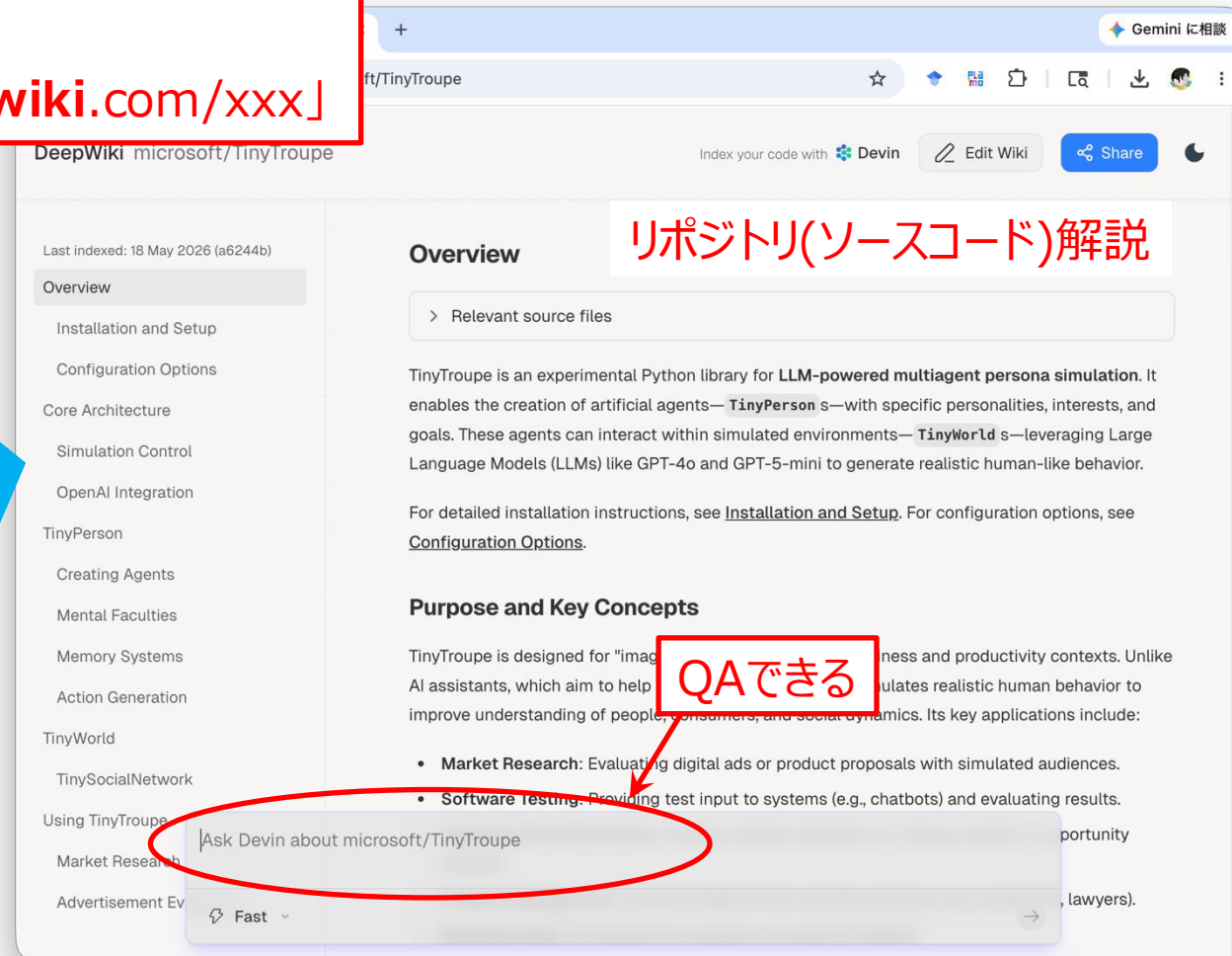
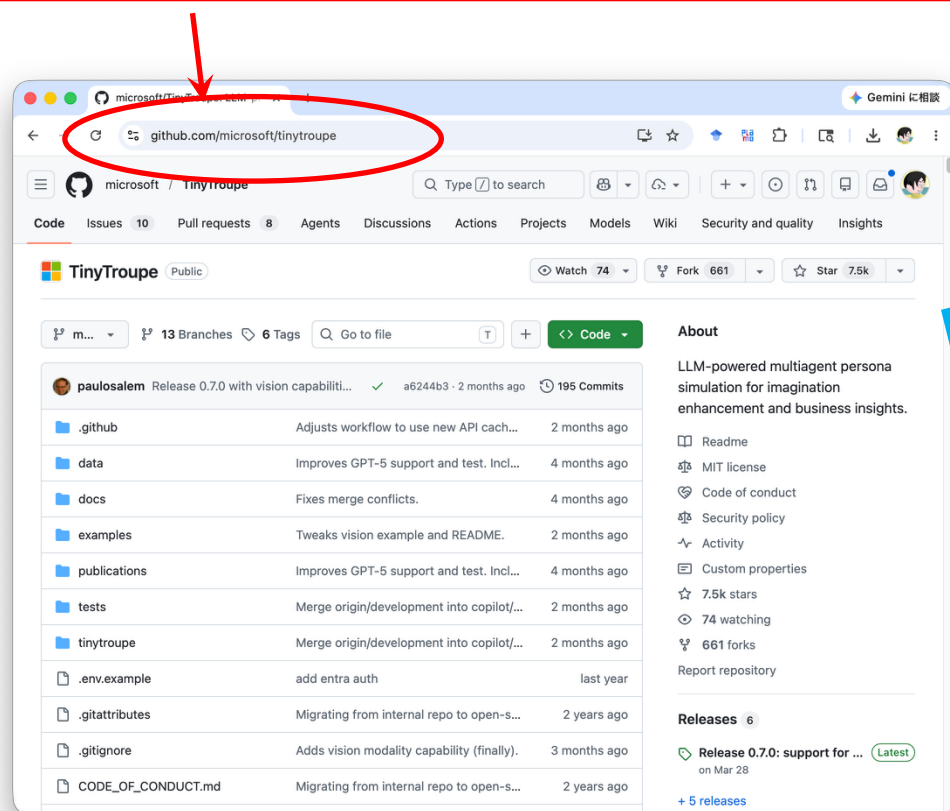
The screenshot shows a web browser window with the URL `chatgpt.com/c/687d9d33-ca18-8011-94fa-08504731782c`. The page displays a summary report titled "楽天トラベル「レジャー」ユーザーコメント分析 (2024/1~2025 GW) 何がどう高(好)評価・低(悪)評価されているか" (Rakuten Travel 'Lejia' User Comment Analysis (2024/1~2025 GW) What is being highly (good) evaluated/low (bad) evaluated?). The report is divided into two main sections: "高(好)評価のポイント" (Points of high (good) evaluation) and "低(悪)評価のポイント" (Points of low (bad) evaluation). Each section contains a table with icons and text describing the evaluation points. A red callout box with a white border contains the text: "今や図も読めるし、図も書ける" (Nowadays, you can read and write diagrams). At the bottom, a summary box states: "温泉・お湯、スタッフ・接客、食事が高く評価される一方、施設の古さ・清潔感・混雑・価格・アクセスが低評価の主な要因に" (While hot springs, staff, and food are highly evaluated, the main factors for low evaluation are the age of the facility, cleanliness, crowding, price, and access).

大規模言語モデル(LLM)、使ってますか？

● 論文実装(ソースコード)の読み方が変わる: [DeepWiki](#)

GitHubリポジトリを開き、URLを直接書き換える

例: 「<https://github.com/xxx>」→「<https://deepwiki.com/xxx>」

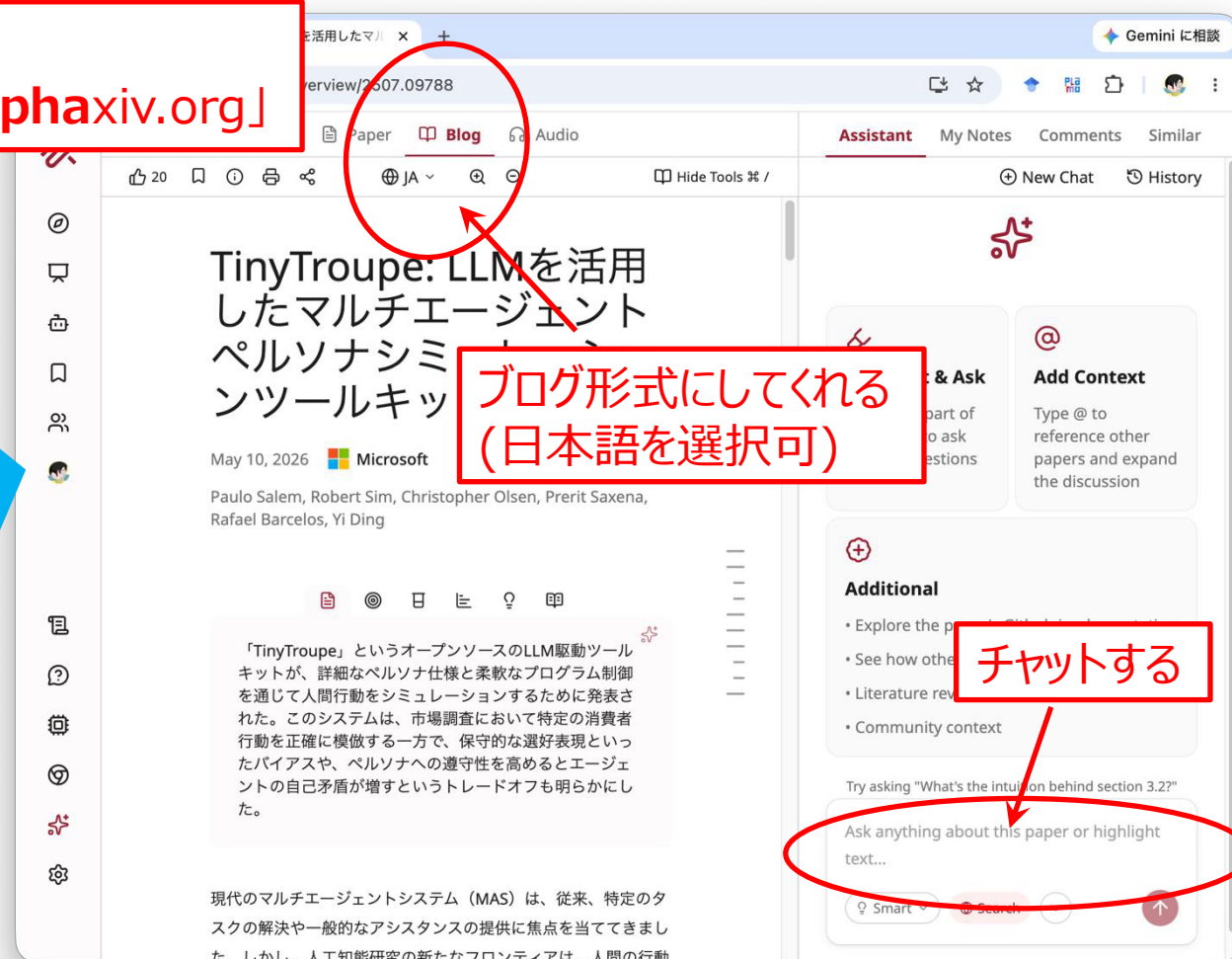
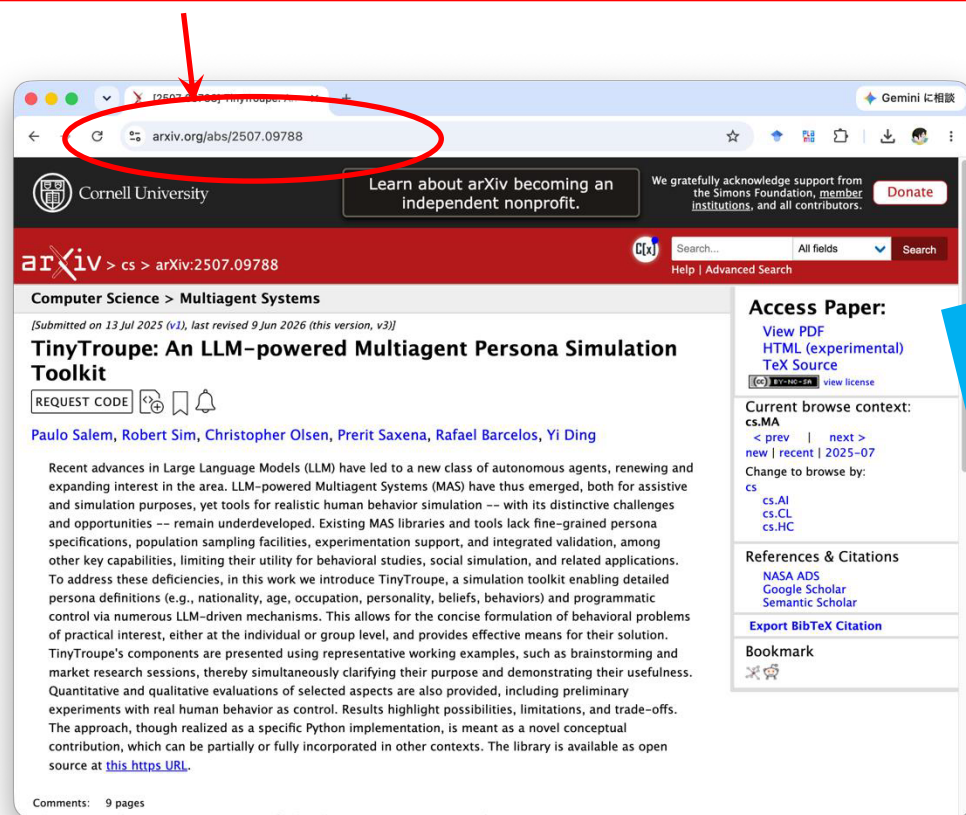


大規模言語モデル(LLM)、使ってますか？

● 論文の読み方が変わる: [alphaxiv](https://www.alphaxiv.org)

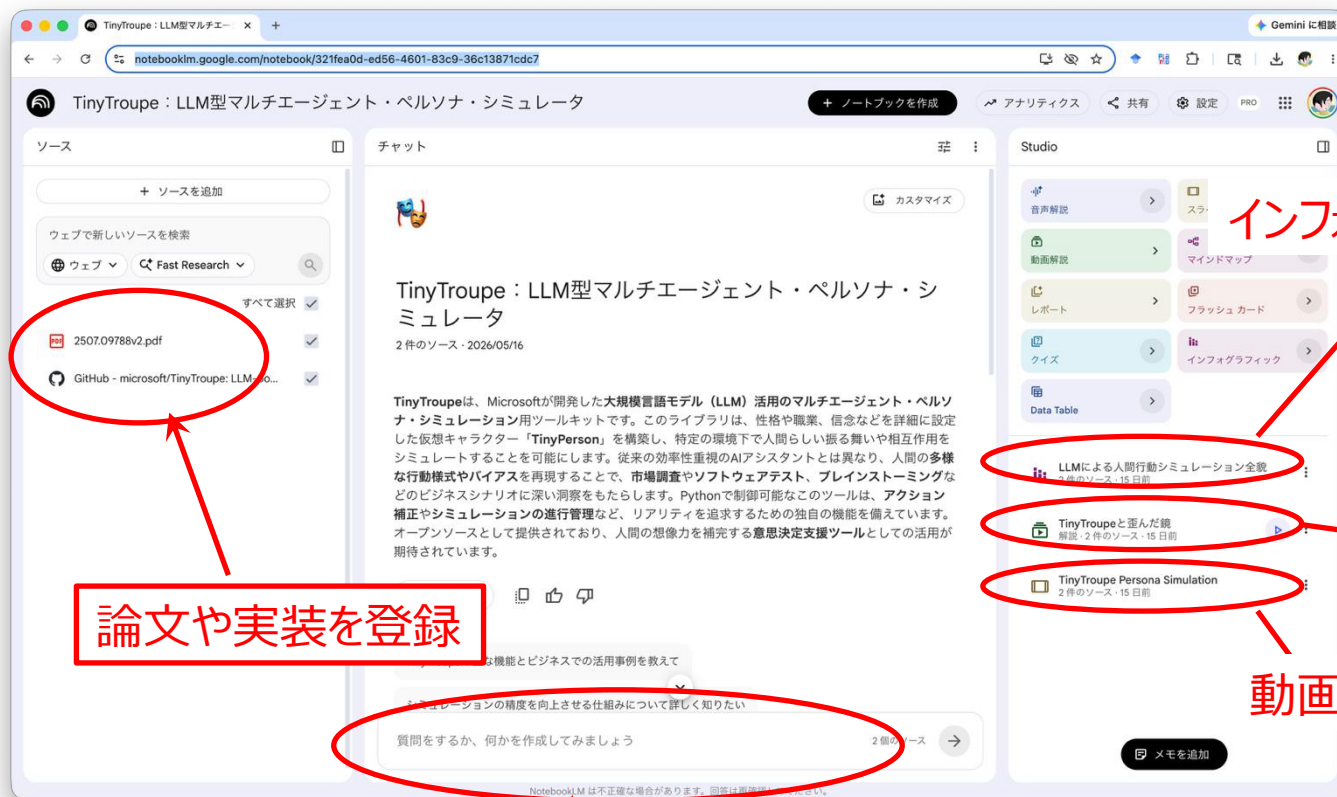
arxiv を開き、URLを直接書き換える

例: 「[https://arxiv.org/xxx](https://arxiv.org/abs/2507.09788)」→「<https://www.alphaxiv.org>」



大規模言語モデル(LLM)、使ってますか？

● 論文の読み方が変わる: [Google NotebookLM](#)



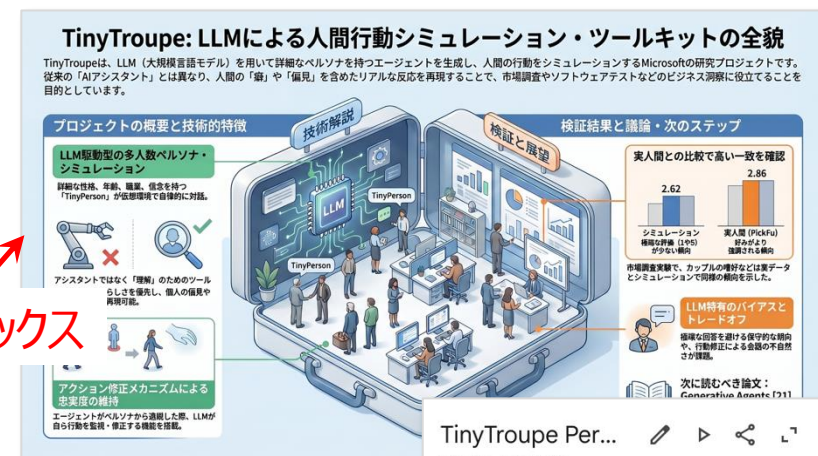
論文や実装を登録

QAする

インフォグラフィックス

動画解説

スライド作成



- AIエージェントの進化: チャットから「仕事を進める」AIへ
 - 単なる回答だけでなく、自分で調べ、計画し、最後までやり遂げるように
 - MCPやスキルが普及し、外部ツールや社内システムを使いこなして、実務を代行
 - 安全に任せるには「できること」と「手順」を正しく伝える「ハーネス」が重要

- AIネイティブ開発: コード補完から自動生成へ



- コード補完から、設計からテストまでを一括支援へ
- 人間が「何を作りたいか」を指示すれば、AIが自動で形にする仕組みが拡大

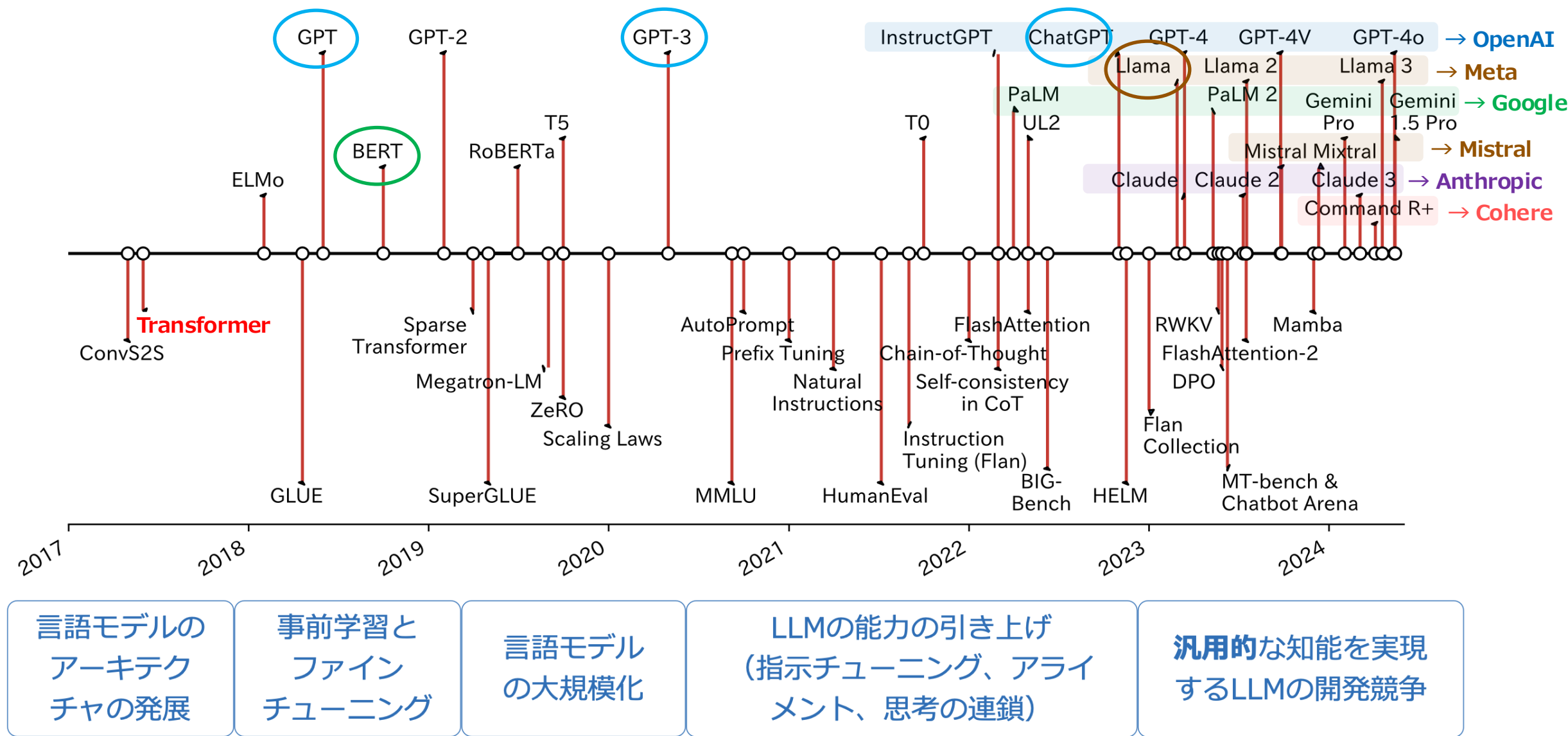
- Physical AI: ロボット知能と世界モデルの統合へ

- 見て・理解して・動く、ロボット向けAIが劇的に進化
- 仮想空間での練習を現実を活かし、学習・検証を効率化
- LLM → VLM → VLA → WAM (世界モデル) へ



自動運転も「Physical AI」の時代へ
出典: [TuringTechTalk#39](#)

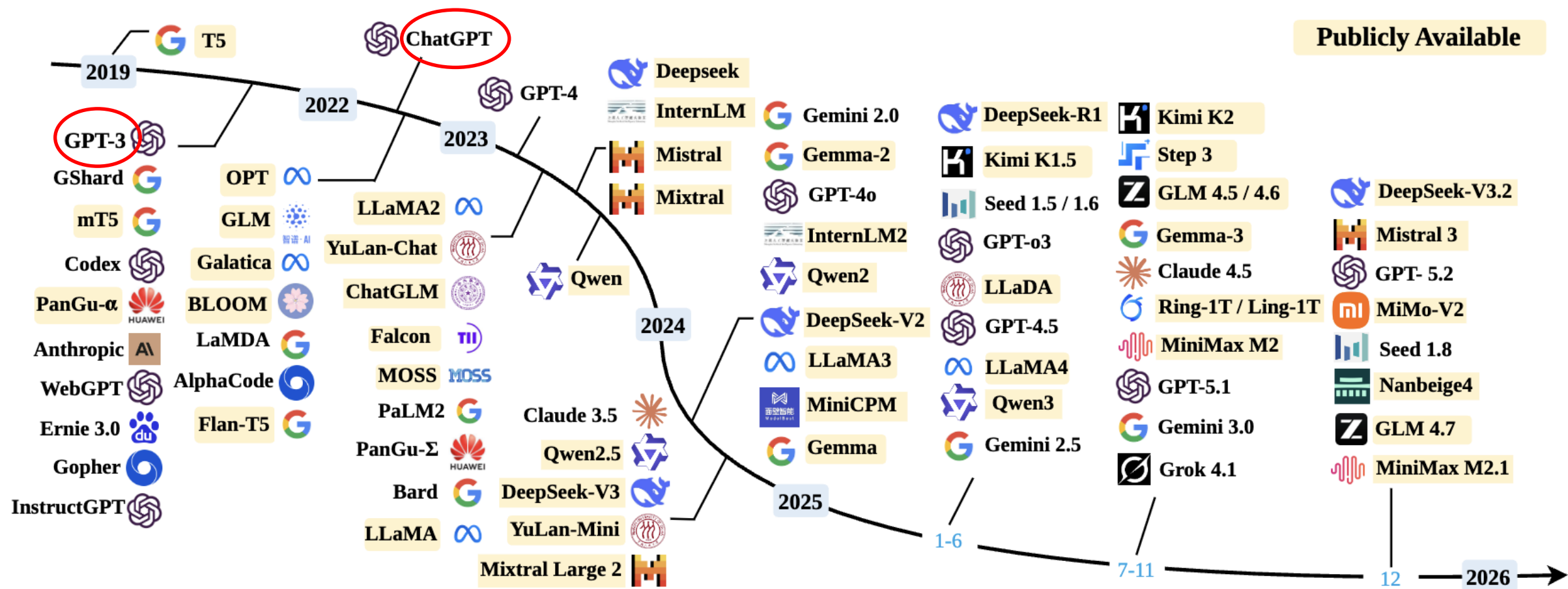
大規模言語モデル(LLM)の歴史 (2017-2024年)



出所: [岡崎, 2024] JSAI2024 チュートリアル講演1「大規模言語モデルの開発」

大規模言語モデル(LLM)の進展

- 2020年のGPT-3登場後、様々な企業から多くのLLMが登場し、技術革新が加速

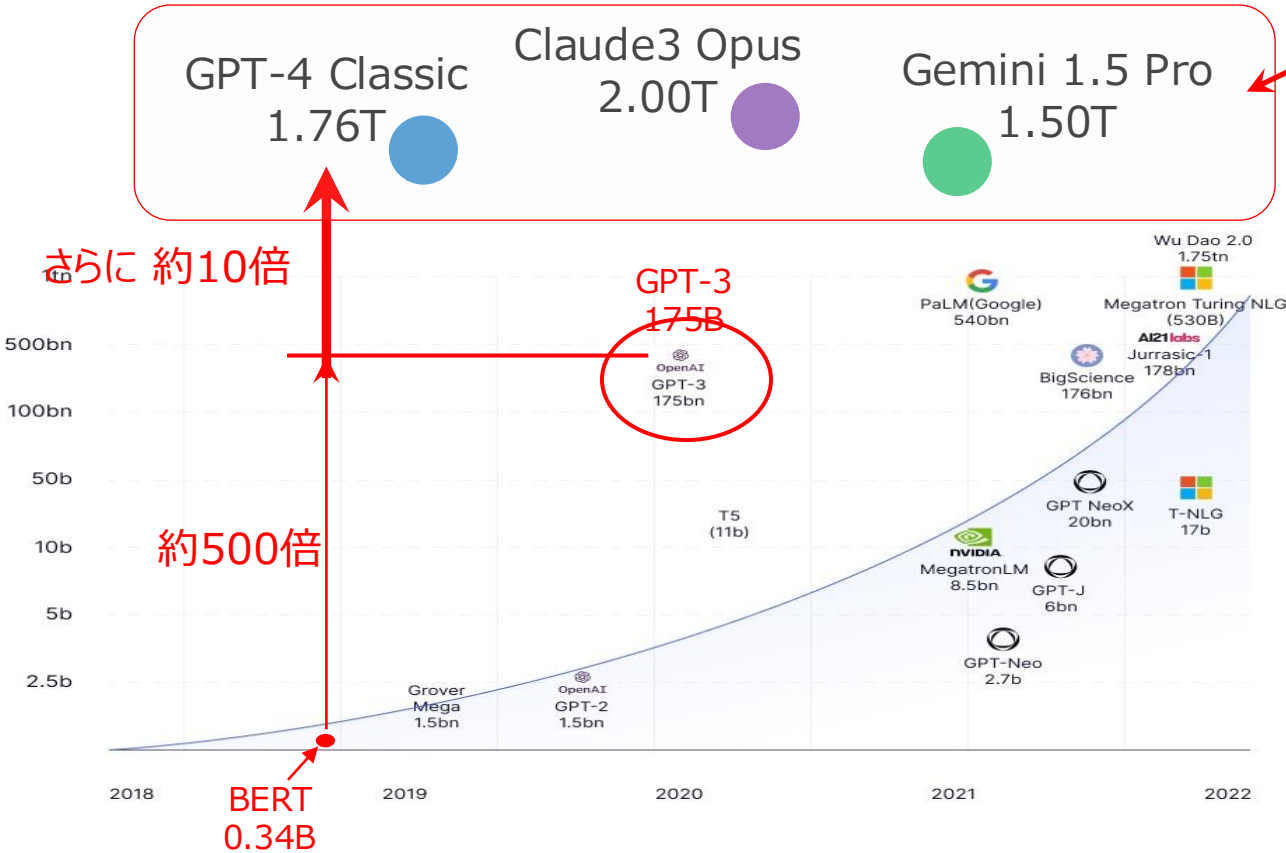


出典: Wayne Xin Zhao et al. (2023), "[A Survey of Large Language Models](#)", [v19]

大規模言語モデル(LLM)は、どのくらい大きいのか

- 大量の計算機資源で、大量データを大きなモデルで学習
"Scaling laws" [Kaplan (OpenAI)+, 2020/0

GPT-5 や Gemini-3、Claude-4 などの
近年のモデルは、正確なパラメータ数は公式に
明らかにされていないものの、数千億～数兆
規模と推測される



Llama 3 70B (Meta) の学習:

パラメタ数	テキストデータ	計算時間 (GPU時間)	計算環境 (GPU数)
700億 (70B)	約15兆トークン (43TB 程度)	6.4M GPU時間 (731 GPU年)	24K 枚 (H100)



AWS H100 の時間単価で 126億円

※p5.48xlarge(8GPUs): \$98.32/h

さくら高火力 PHY の月額単価で 34億円

※高火力PHY(8GPUs): ¥3,046,120/月

出典: <https://textcortex.com/ja/post/how-gpt-3-writing-tools-work>

出典: [Llama 3 のモデルカード](#)、[Andrej Karpathy 氏のX](#)

大規模言語モデル(LLM)は、どのくらい大きいのか

- AI は巨大なデータセンタで作られている → 百メガワット~ギガワット



左図から試算:

1つのモデルを数ヶ月で作る



AWS H100 の時間単価で 122億円

※p5.48xlarge(8GPUs): \$98.32/h

さくら高火力 PHY の月額単価で 31億円

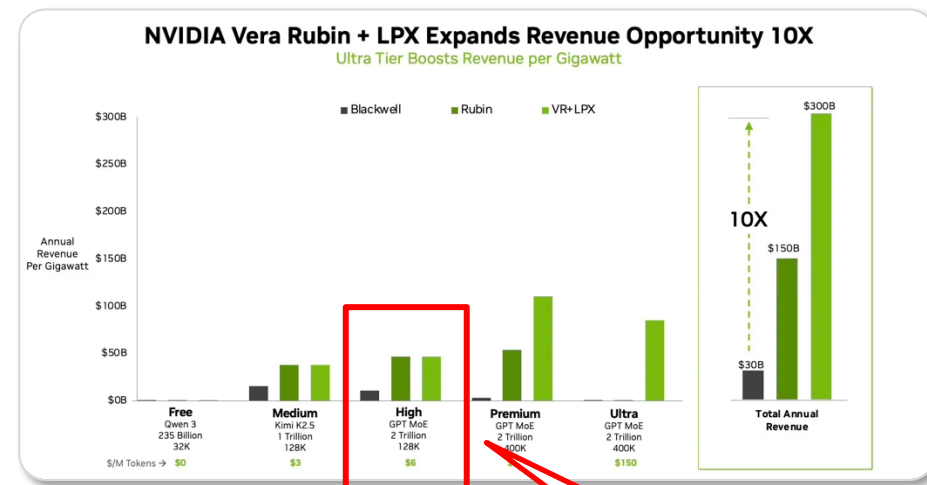
※高火力PHY(8GPUs): ¥3,046,120/月

- データ収集~学習~メンテナンス → 100人~数千人のチームを24時間組んで働く工場

出典: [Grok3 Launch @elonmusk](#)

大規模言語モデル(LLM)は、どのくらい大きいのか

- AIデータセンタ = トークン工場
 - Input:
電力、施設、AIサーバー、AIモデル
 - Outcome:
トークン (1Mトークンあたり\$0.2~\$25)



出典: [NVIDIA Technical Blog](#)

1GWで年間\$50B
(7.5兆円)※の売上を
作れるポテンシャル

- 大規模AIサービスでは、数十兆~百兆トークン/日の規模
 - Google 2026年5月時点で3200兆トークン/月 (Google I/O)
- 史上最大規模の計算需要で、今後も増加し続けると予想される
 - ソフト開発支援が中心 →生活や業務で普及フェーズに
 - AIEージェントがさらに利用を加速 →年率10倍ペース

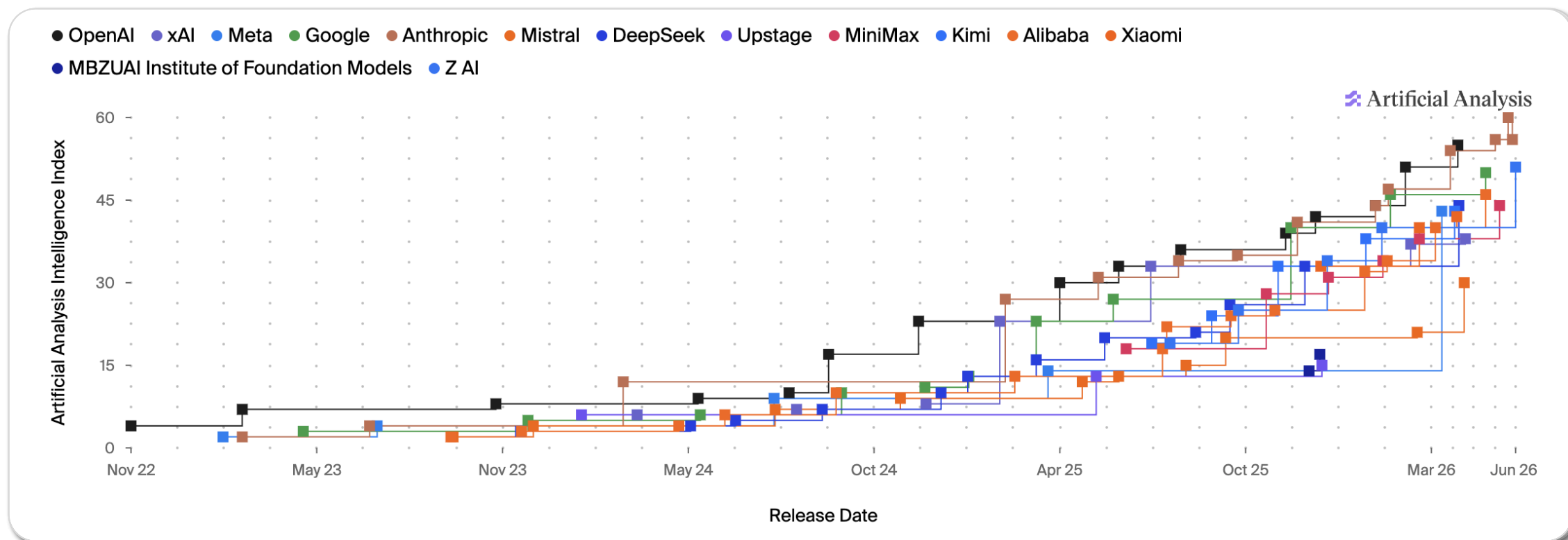
※ トークン/日を試算:
 $\$50\text{B}/\text{年} \div \$6 \times 1\text{M}\text{トークン} = 8333\text{兆}\text{トークン}/\text{年}$
 $8333\text{兆} \div 365\text{日} = 23\text{兆}\text{トークン}/\text{日}$

出典: [岡野原'26] JSAI2026 特別講演「AIの現在地と次の挑戦: 科学・工学・社会の交点から」

大規模言語モデル(LLM)は、どのくらい賢いか

- 2023年に Google Gemini Ultra が数学、歴史、情報科学、法律などの57の領域の問題で構成されるベンチマークで、**人間の専門家**によるスコア (89.8%) を超えた
- その後も、様々なタスクが解けるようになり、淡々と少しずつ賢くなっている

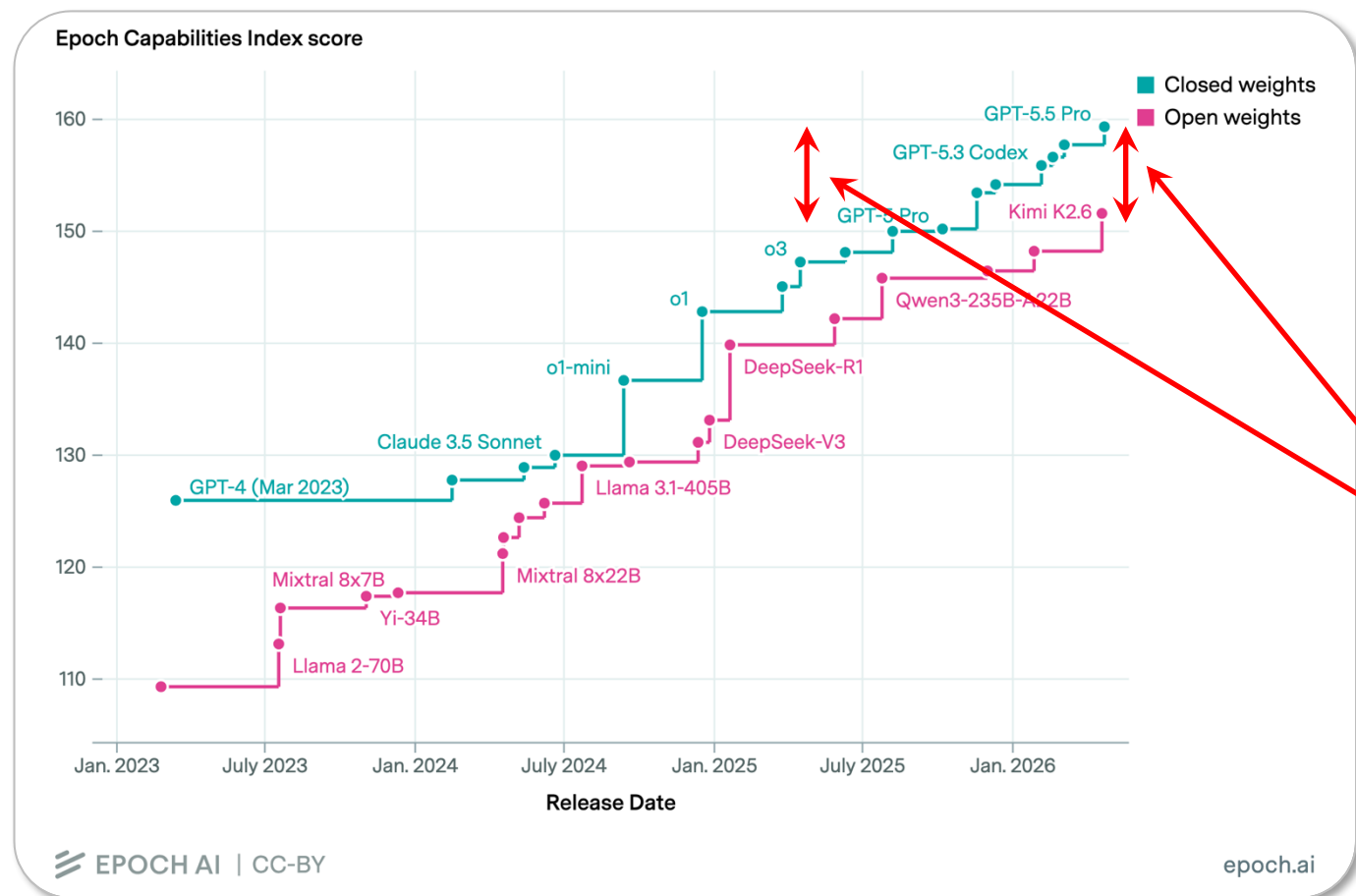
※ Artificial Analysis Intelligence Index: 様々なタスクの加重平均



出典:「[Artificial Analysis Intelligence Index](#)」(Artificial Analysis)

オープン大規模言語モデル(LLM)

- 2023年はじめ頃から短い期間で性能が大きく改善
- **オープン(緑)**と**クローズド(赤)**との性能差が狭くなってきている → LLMの“民主化”が進む



優れたオープンモデルは、性能や学習計算でクローズドモデルにほぼ追いついており、2024年は約1年あった差が今では約4か月に縮まっている

高性能なオープンモデルと最先端のクローズドモデルの差はGPT-5とGPT-5.5の差とほぼ同じ

出典: 「[Open models lag state-of-the-art closed models by 4 months](#)」 (EPOCH AI)

オープン大規模言語モデル(LLM) — 代表的なオープンウェイトモデル

● 大型(≧30B)のオープンウェイトモデル

開発元	モデル名	ライセンス
Alibaba Cloud	Qwen3.5-397B-A17B	Apache 2.0
Alibaba Cloud	Qwen3.5-122B-A10B	Apache 2.0
Google	gemma-4-31b-it	Gemma
Alibaba Cloud	Qwen3.5-35B-A3B	Apache 2.0
DeepSeek	DeepSeek-V3.2	DeepSeek
Z.ai	GLM-5	MIT
Moonshot AI	Kimi-K2.5	Kim
Alibaba Cloud	Qwen3-235B-A22B-Thinking-2507	Apache 2.0
Alibaba Cloud	Qwen3-Next-80B-A3B-Thinking	Apache 2.0
MiniMax	MiniMax-M2.1	
Alibaba Cloud	Qwen3-30B-A3	
LG AI Research	K-EXAONE-236	
Alibaba Cloud	Qwen3-Next-80B-A3B-Instruct	Apache 2.0
OpenAI	gpt-oss-120b	Apache 2.0

10B以下は「SLM」と呼ばれ
一般的なPCでも動作可能
(但しCPUだとかなり遅い)

● 小型(<30B)のオープンウェイトモデル

開発元	モデル名 (国内)	ライセンス
Google	gemma-4-26B-A4B-it	Gemma
Alibaba Cloud	Qwen3.5-27B	Apache 2.0
Alibaba Cloud	Qwen3.5-9B	Apache 2.0
Alibaba Cloud	Qwen3-14B	Apache 2.0
東京科学大学	GPT-OSS-Swallow-20B-RL-v0.1	Apache 2.0
東京科学大学	Gemma-2-Llama-Swallow-27b-it-v0.1	Apache 2.0
Alibaba Cloud	Qwen3.5-4B	Apache 2.0
NVIDIA	NVIDIA-Nemotron-Nano-9B-v2-Japanese	NVIDIA
Alibaba Cloud	Qwen3-VL-8B-Thinking	Apache 2.0
Alibaba Cloud	Qwen3-8B	Apache 2.0
Google	gemma-4-E4B-it	Gemma
Google	gemma-4-E2B-it	Gemma
東京科学大学	Qwen3-Swallow-8B-RL-v0.2	Apache 2.0
東京科学大学	Gemma-2-Llama-Swallow-9b-it-v0.1	Apache 2.0

出典: 「[Nejumi LLMリーダーボード4](#) ランキング上位 (2026/6/23時点)」 の加筆して修正

自然言語処理はどうなる？

- 人間が書いたり話したりする言葉をコンピュータで処理する技術・研究分野

- 応用技術：自然言語処理を応用したアプリケーション

テキスト検索 テキスト分類 テキスト要約 情報抽出

機械翻訳

質問応答

**ChatGPTで
いいんじゃない？**

など

- 基盤技術：言語を応用タスクで利用しやすい形式に変換する

言語モデル

形態素解析

固有表現抽出

構文解析

意味解析

文脈解析

談話解析

など

大規模言語モデル (LLM) の成り立ち

(再掲) 自然言語処理とは

- 人間が書いたり話したりする言葉をコンピュータで処理する技術・研究分野
 - 応用技術: 自然言語処理を応用したアプリケーション

テキスト検索 テキスト分類 テキスト要約 情報抽出
機械翻訳 質問応答 対話 など

- 基盤技術: 自然言語を応用タスクで利用しやすい形式に変換する

言語モデル 形態素解析 固有表現抽出 構文解析
意味解析 文脈解析 談話解析 など

言語モデル (Language Model: LM) とは

- 次に来る言葉を「確率」で予想する仕組み (確率的言語モデル)

単語の並び $\Rightarrow y_1, y_2, \dots, y_T$ 生成確率 $\Rightarrow P(y_1, y_2, \dots, y_T)$

- 具体的な例: 「日本の首都は...」の後に続く言葉は?

$$y^* = \operatorname{argmax}_{y \in V} P(\text{日本, の, 首都, は, } y)$$

$P(\text{日本, の, 首都, は, ロンドン}) = 0.00000043$

$P(\text{日本, の, 首都, は, パリ}) = 0.00000082$

$P(\text{日本, の, 首都, は, 東京}) = 0.00000103$

$P(\text{日本, の, 首都, は, } \dots) = \dots$

一番確率が高い言葉を
自動で選ぶ



東京

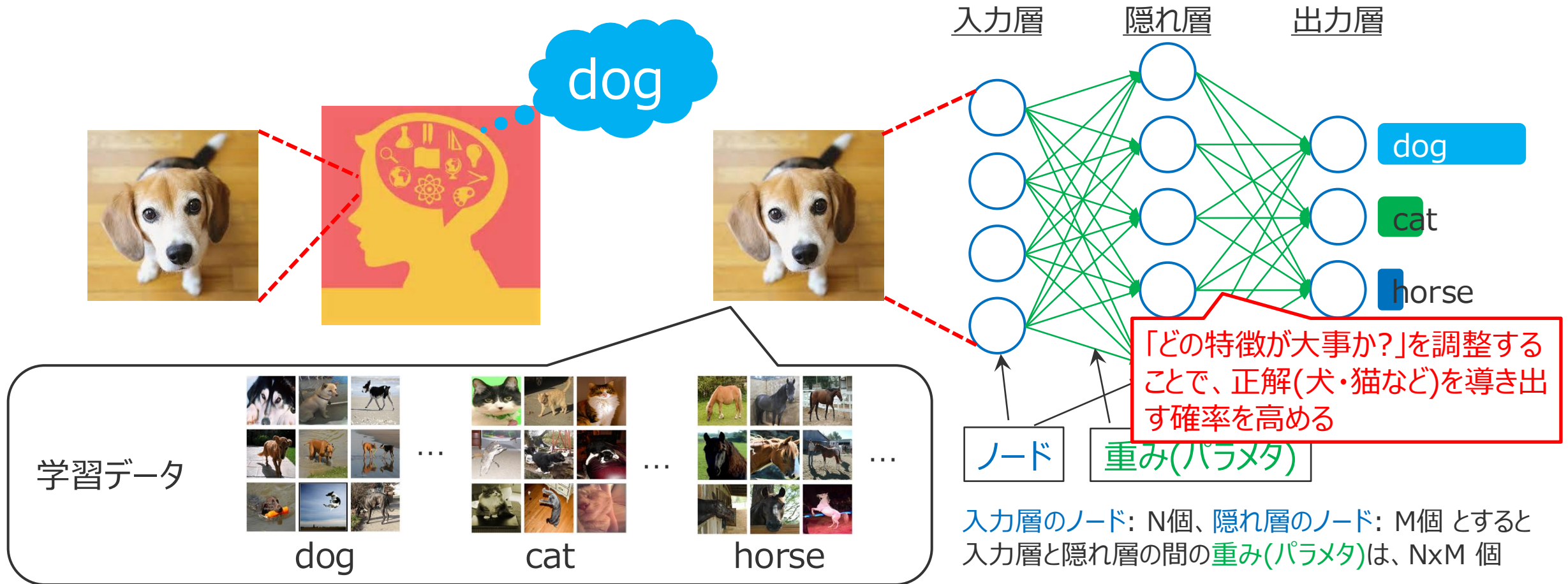
- 単語の並び方を丸ごと覚え、前後のつながり(文脈)も考慮 $\rightarrow$ N-gram言語モデル

$$y^* = \operatorname{argmax}_{y \in V} P(\text{日本, の, 首都, は, } y) = \operatorname{argmax}_{y \in V} P(y | \underbrace{\text{日本, の, 首都, は}}_{\text{yの前方N-1個の単語(トークン)}})$$

yの前方N-1個の単語(トークン)

ニューラルネットワークとは

- 人間の脳(神経細胞)の働きをマネして、コンピュータに学習(機械学習)させる方法
 - 「機械学習」とは大量のデータを学習して特徴(パラメータ)を獲得すること



機械はどうやって単語を理解する? (単語ベクトル)

- 分布仮説 [[Harris+,1954](#)]

- 周りにある単語からその言葉の意味が推測できるという仮説

→ 似た使われ方をする言葉は、意味も似ている

文1: 昨日、りんごを食べた。りんごジュースを飲んだ。りんごの皮をむいた。

文2: 昨日、りんごを食べた。ぶどうジュースを買った。ぶどうの皮をむいた。

文3: 昨日、自転車に乗った。自転車を修理した。自転車を買った。

共起による
ベクトル表現
[[Lin,2002](#)]

	...	食べる	...	飲む	...	修理	...
りんご	0	1	0	1	0	0	0
ぶどう	0	1	0	1	0	0	0
自転車	0	0	0	0	0	1	0
:							

似てる
似てない

← 語彙数(数万～数十万)分の疎なベクトルになる →

機械はどうやって単語を理解する? (単語ベクトル)

- 分布仮説 [[Harris+,1954](#)]

- 周りにある単語からその言葉の意味が推測できるという仮説

→ 似た使われ方(文脈)をする言葉は、意味も似ている

- 意味をたくさんの要素に分解する (=分散表現)

→ 少なめの要素(例えば100次元)で効率よく表現、値は実数値

単語埋め込み
(word embedding)
とも呼ばれる

これらの実数値を **ニューラルネットワーク** で割り当てる

共起による
ベクトル表現
[[Lin,2002](#)]

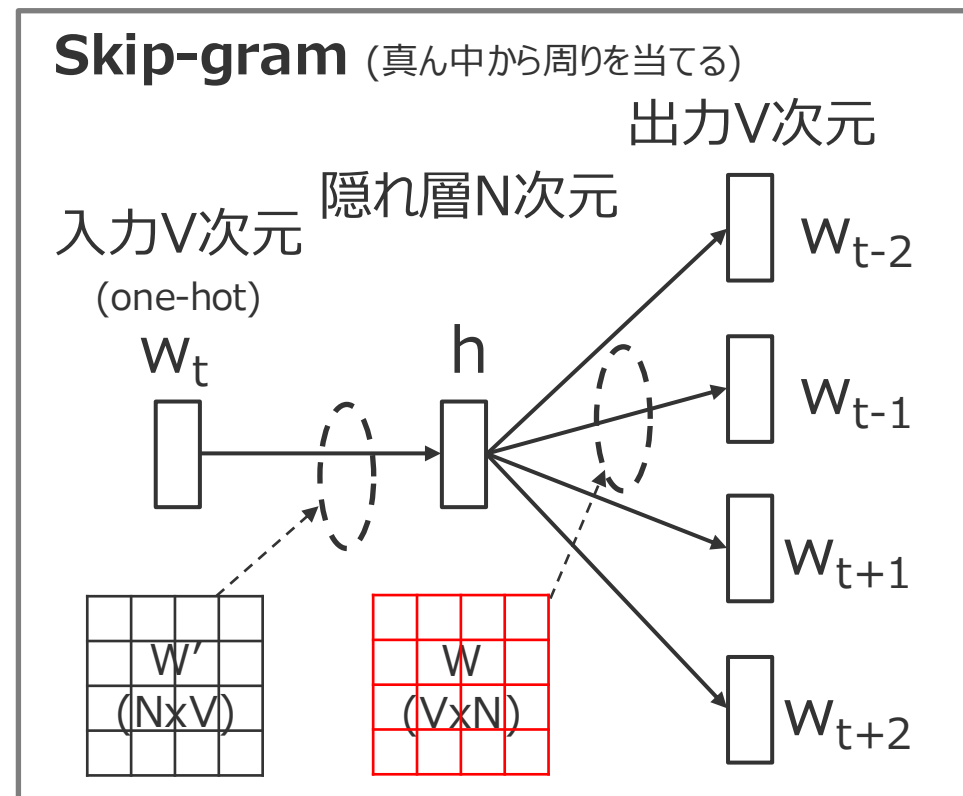
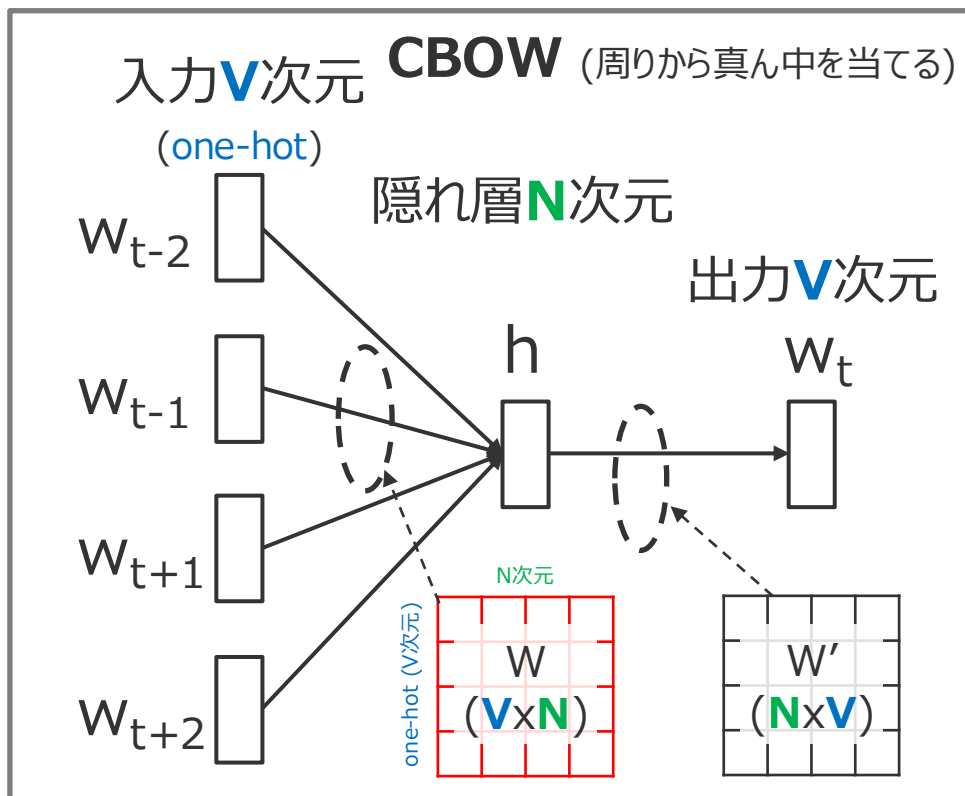
次元→	0	1	...	50	...	98	99
りんご	1.07	-1.08		1.48		0.46	0.48
ぶどう	1.95	-1.53		0.36		-0.61	-0.44
自転車	0.67	1.44		-1.50		0.10	0.67
:							

似てる
似てない

← **100次元**程度の要素でギュッと凝縮 →

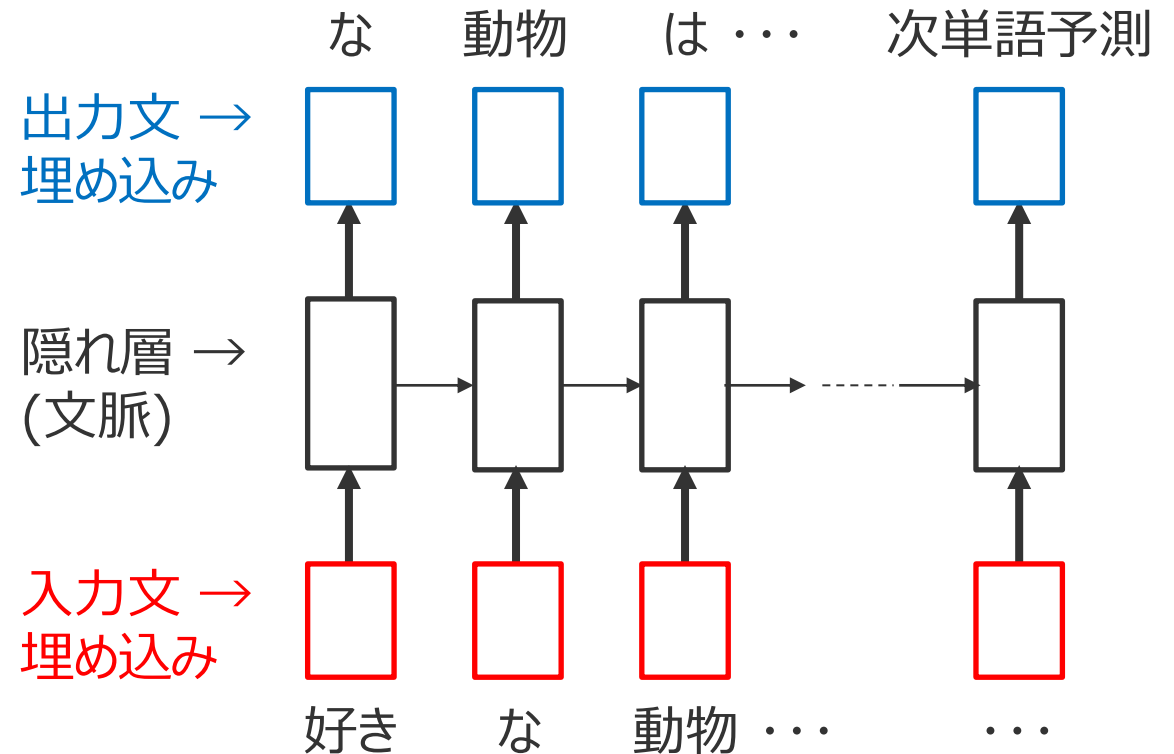
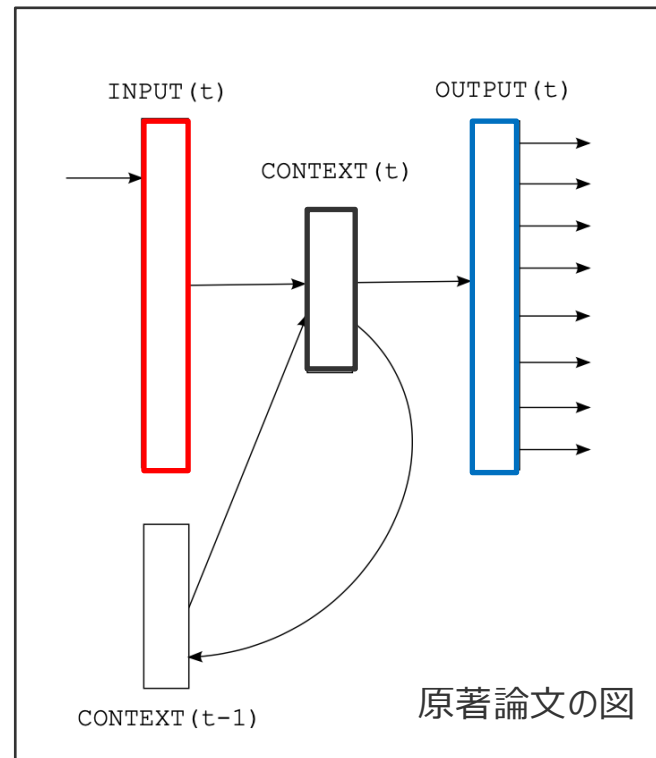
機械はどうやって単語を理解する? (単語ベクトル)

- 代表格: word2vec [[Mikolov+, 2013](#)]
 - これまでも「似ているかどうか」の比較はできたが、**足したり引いたり**できなかった
 - king - man + woman = queen が有名



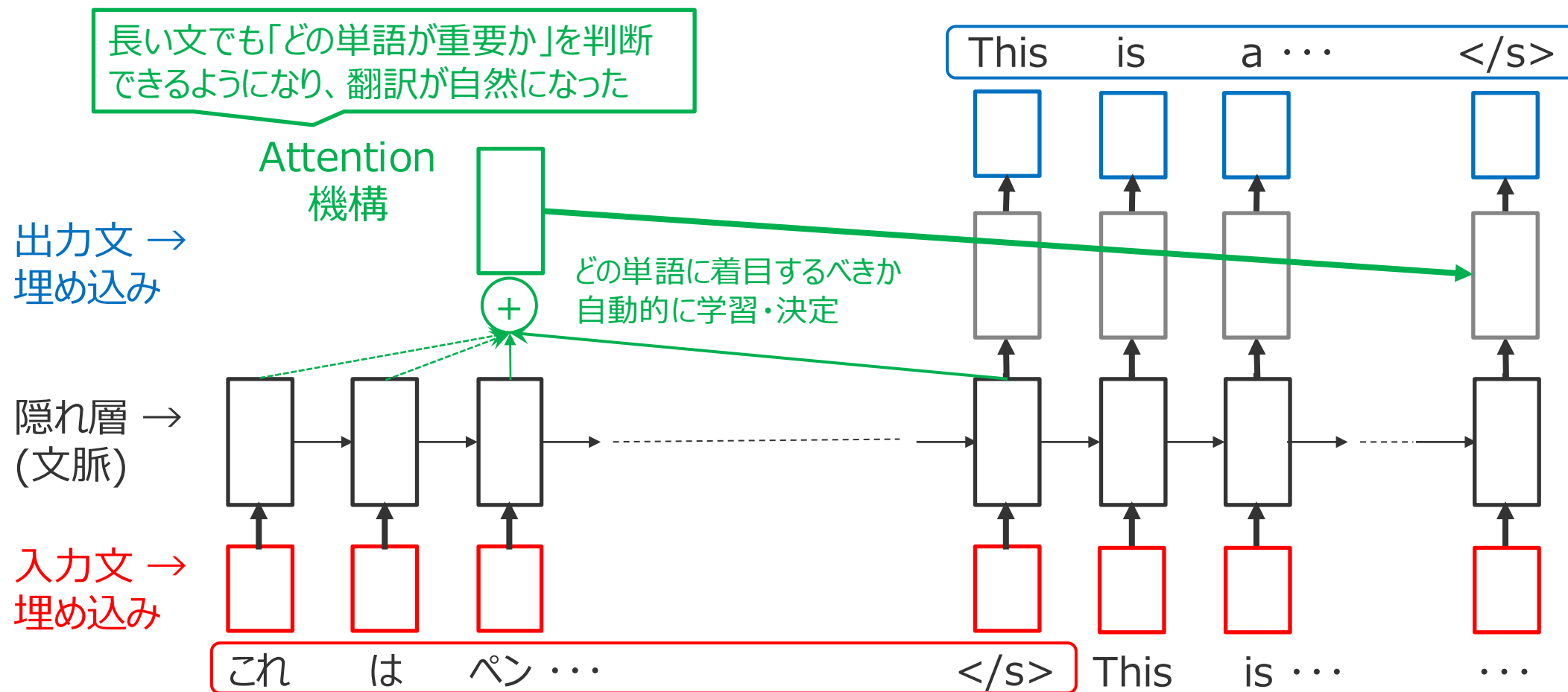
● RNN 言語モデル [[Mikolov+,2010](#)]

- これまでの言葉の流れを「記憶」して、次にくる言葉を予想する仕組み
- 文章を読み進めるほど、それまでの「文脈」がモデルの中(隠れ層)に蓄積されていく



エンコーダーデコーダー型の機械翻訳モデル

- Seq2Seq [[Sutskever+, NIPS2014](#)]: ニューラル機械翻訳の基本となったモデル
- Attention 機構 [[Bahdanau+, 2015](#)] により、長い文の翻訳性能が向上

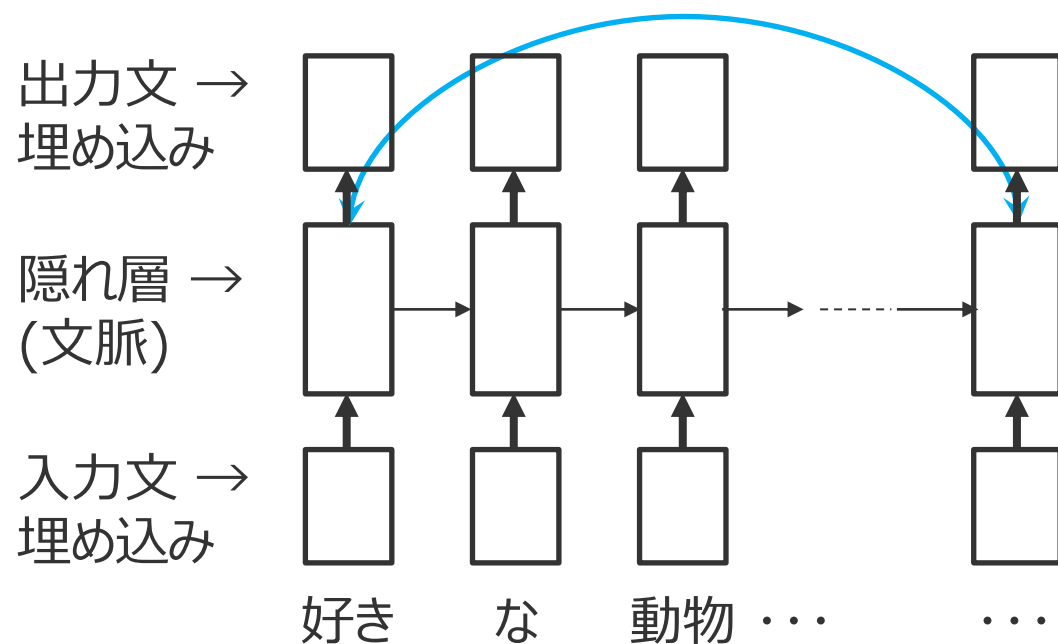


Self-Attention の登場

- 従来来の仕組み: 長い文章だと、最初の方に出てきた言葉を忘れてしまいがち
- Self-Attention: 離れた場所の言葉同士の「つながり」を見つけ、文の理解度が向上

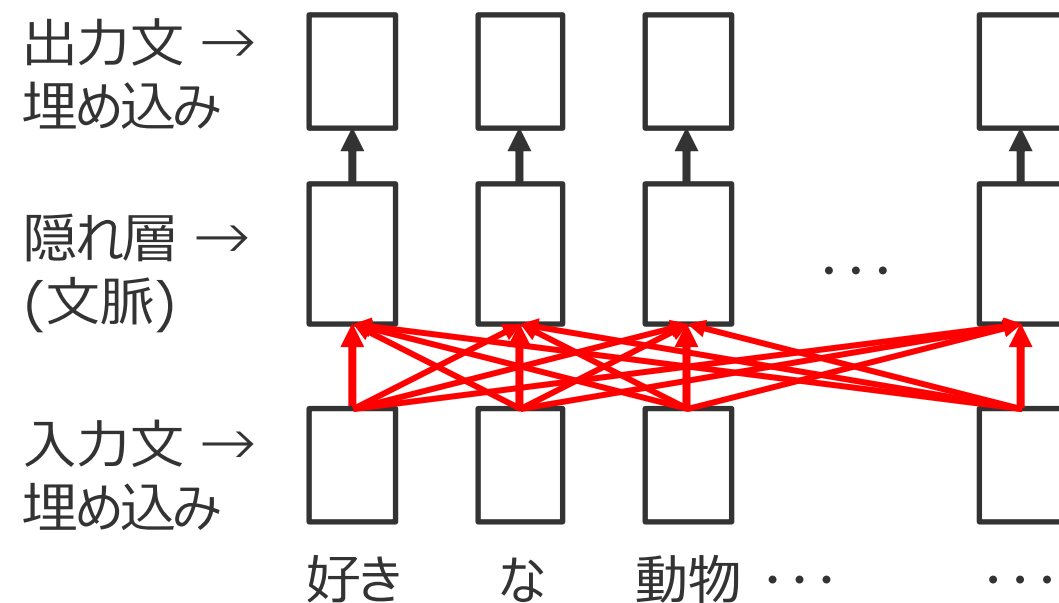
従来(LSTM)

遠く離れた単語の関係性を
捕まえにくい



Self-Attention

遠く離れた単語も直接
関係性を考慮できる



Transformer [[Vaswani+, NIPS2017](#)]

- 単語間の関係性を「アテンション」のみで計算※することで、翻訳精度が飛躍的に向上し、機械翻訳で圧倒的な SOTA を達成 (※論文名の「Attention Is All You Need」が有名)

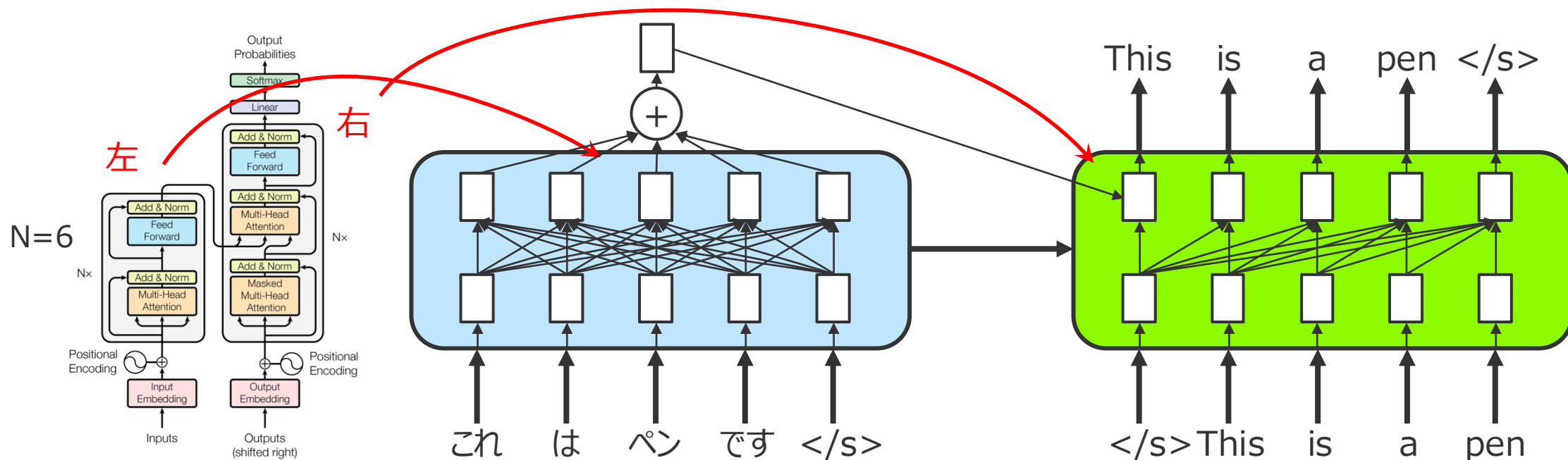


Figure 1: The Transformer - model architecture.

よく見る図

Transformer **エンコーダ**

Transformer **デコーダ**

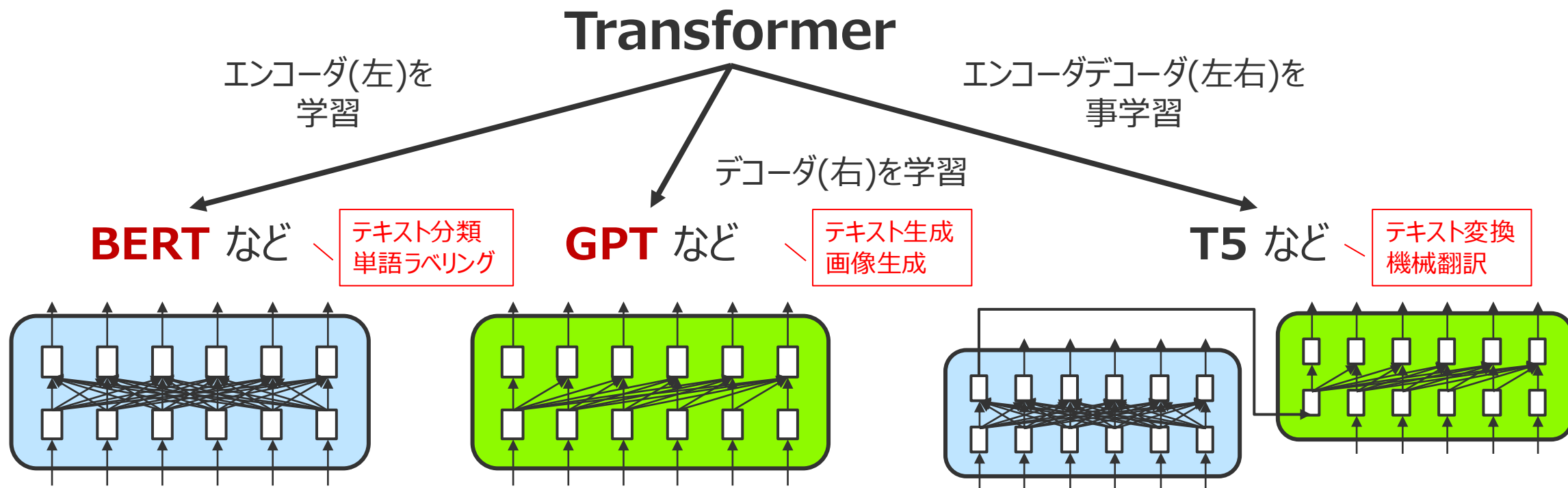
引用: [[西田, 2022](#)] JSAI2022 チュートリアル講演資料の一部を修正して作成

双方向型のNNモデル
(右側の単語も使う)

自己回帰型のNNモデル
(出力を入力に戻す)

Transformer [[Vaswani+, NIPS2017](#)]

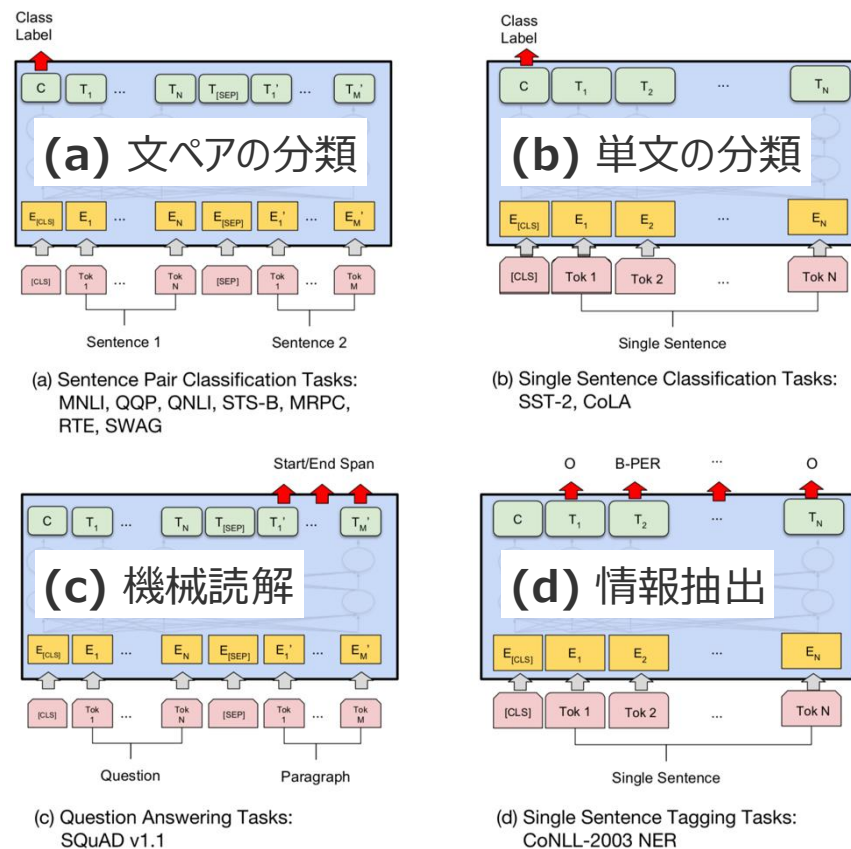
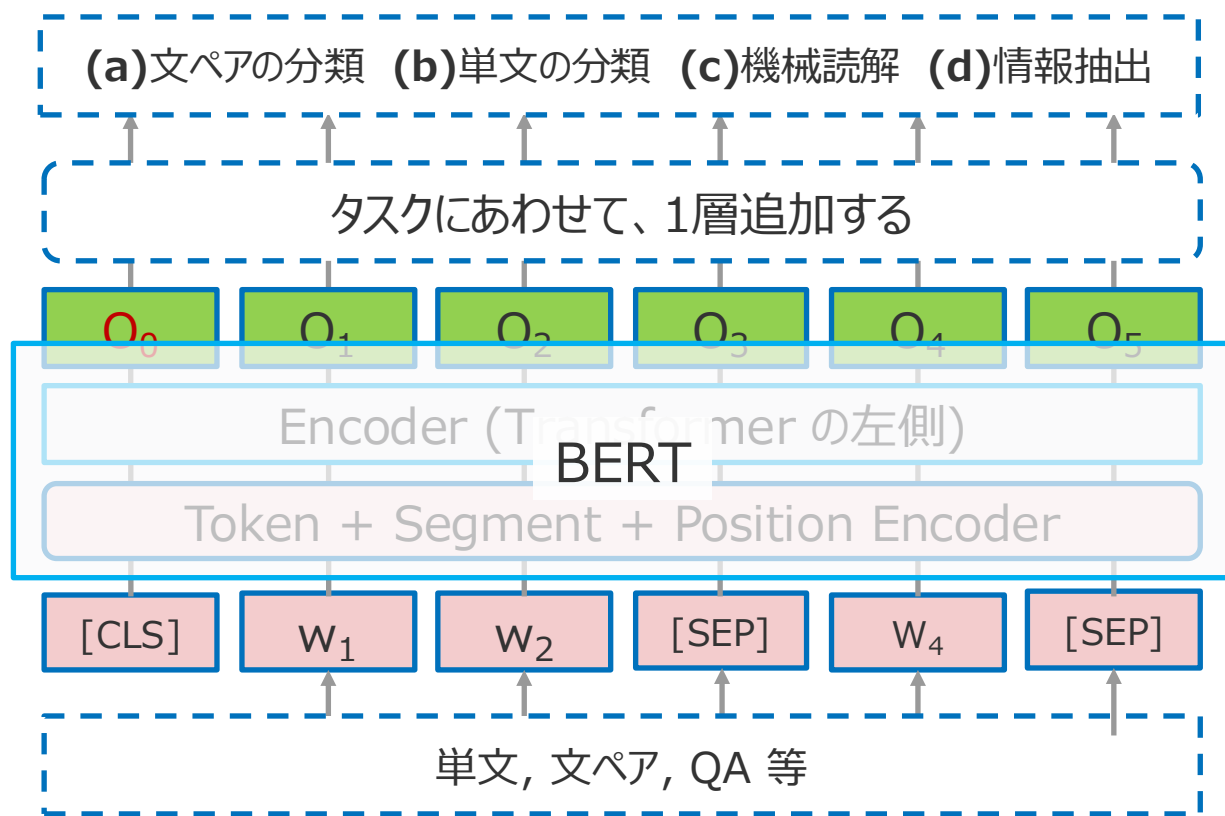
- 最近のほとんどのAIモデルには、Transformer の技術が使われている
- 言語に限らず、画像や動画の世界でも Transformer が高い性能を発揮している



引用: [[西田, 2022](#)] JSAI2022 チュートリアル講演資料の一部を修正して作成

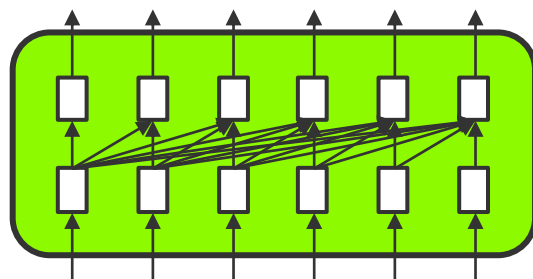
BERT [[Devlin+ \(Google\), NAACL19](#)]

- 機械読解で人間のスコア超え、多数の言語処理タスクで高性能を出し、注目を集めた
 - Transformer(双方向)のブロックを24層も重ねた言語モデル
 - 翻訳や分類などの目的のタスクの層を1層のみ追加して、様々なタスクに適応可能



GPT-3 [[Brown+ \(OpenAI\), NeurIPS2020](#)]

- GPT-1(1億パラメタ),GPT-2(15億パラメタ)と同じモデルで、規模が桁違いに大きい
- 超大量(3000億トークン)のテキストを超巨大(96層)の Transformer デコーダで1750億※のパラメタを学習 (例: BERTは、3.3億トークン,24層,3.4億パラメタ)
- やりたいことの指示(プロンプト)を文章で入力するだけで、様々なタスクを実現
 - Few-shot: 数個(10から100)の例と、指示を文章で与える
 - One-shot: 1個の例と、指示を文章で与える
 - Zero-shot: 指示のみ文章で与え、全く例を与えない



Transformer **デコーダ**

自己回帰型のNNモデル
(出力を入力に戻す)

※ 2022年4月に Google が公開した PaLM は 5400億 [[Chowdhery+,2022](#)]

GPT-3 [[Brown+ \(OpenAI\), NeurIPS2020](#)]

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

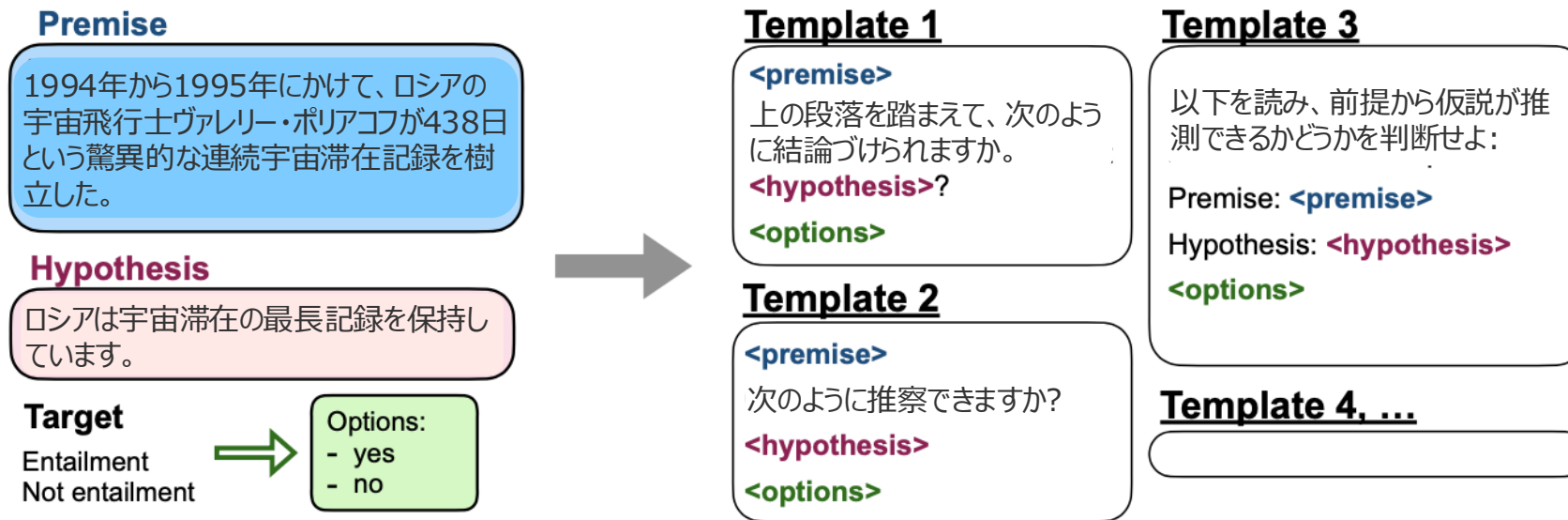
```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

In-context learning:

大量のデータで学び直す必要はなく、
指示文(プロンプト)の中だけで「やり方」を
理解して動く能力のこと

Instruction Tuning (FLAN) [[Wei+, 2022](https://arxiv.org/pdf/2109.01652.pdf)]

- GPT-3 などのモデルの構造は変えずに、**複数のタスク**をファインチューニングする方法
- 従来のデータセットから「指示の出し方」を工夫したテキストを作り、とにかく「指示通り答える」を覚え込ませる (=Instruction Tuning)
 - 例を見せない(ゼロショット)タスクでも、GPT-3よりも高い精度を達成
 - これまでに見たことのない問題に対しても、柔軟に回答



図：前提文が仮説を含意するか否かを自動判定するタスク(NLI=自然言語推論タスク)の例

引用: <https://arxiv.org/pdf/2109.01652.pdf>

ChatGPT (2022/11/30) — 大規模基盤モデル GPT-3 をファインチューニングしたモデル

物知りな GPT-3 に「**話し方のコツ(Instruction Tuning)**」と「**人間の好みの答え方(RLHF)**」を教え込み、自然な対話ができるように調整(ファインチューニング)したモデル

Step1

人間の用意した望ましい回答で事前学習モデル(GPT-3)を fine-tuning

A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3.5 with supervised learning.

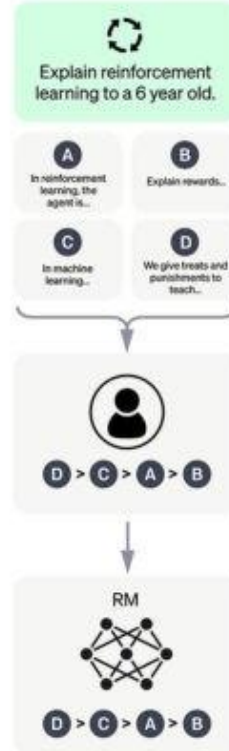
Step2

Step1の出力に人間がランク付けし、報酬モデルを学習

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step3

Step2で学習した報酬モデルを使ってPPOで強化学習

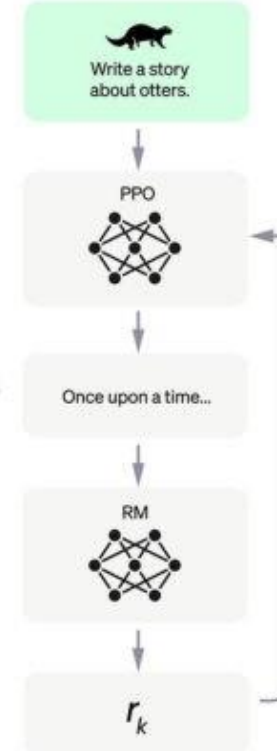
A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Instruction Tuning:

「質問に答える」を練習させ、指示通りに動くようにする方法 (→前頁)

RLHF: 人間からの「こっちの方が良い」という評価を元に学習(強化学習)する方法

1. 人間がモデルの出力を採点する
2. 高得点を狙うコツ(報酬モデル)を学習
3. 報酬モデルを使って、人間好みの話し方を習得(=強化学習)

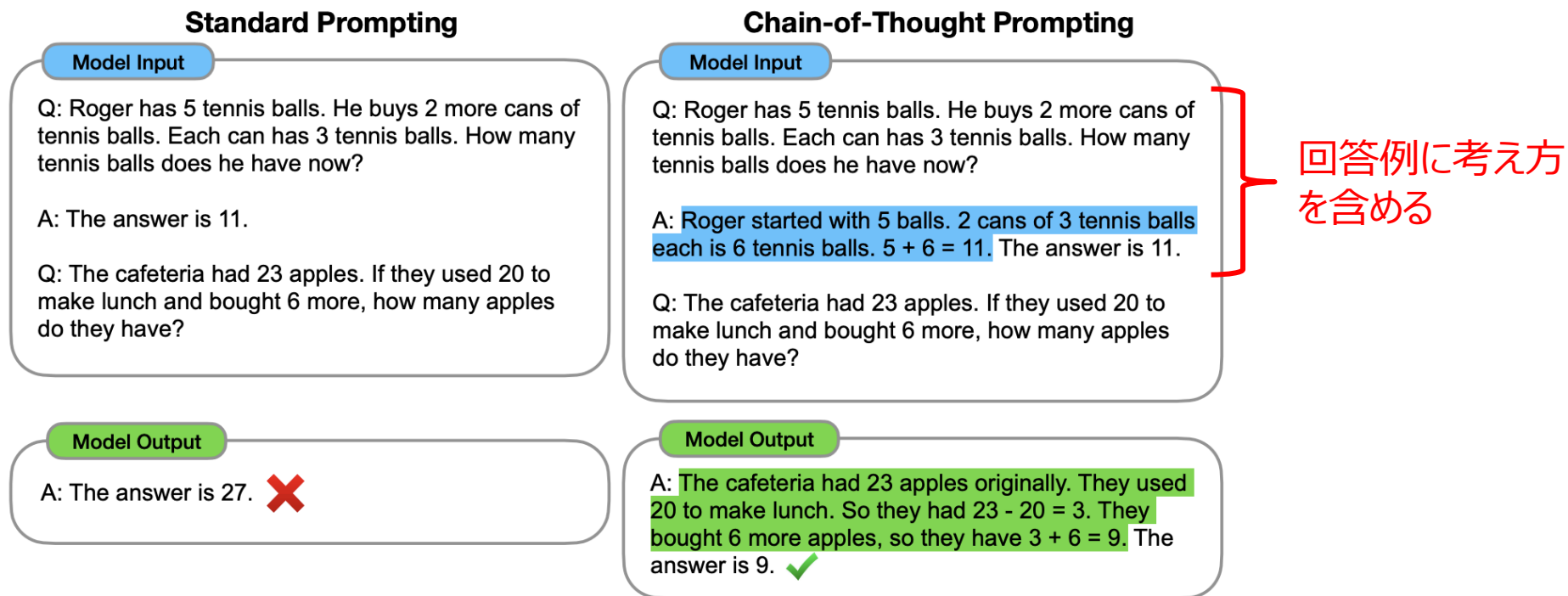
引用: <https://openai.com/blog/chatgpt/>

- 様々な手法やTipsが溢れている → 論文などで効果が検証された手法を使うのが良い

Few-shot Learning	「こう答える」という例をいくつか見せて、やり方を覚えさせる基本的な手法
Chain of Thought (CoT)	いきなり答えを出さず、途中の考え方も書かせることで、難しい問題でもミスを防ぐ手法
Self-consistency	同じ問題を何度か解かせてみて、一番多かった答えを採用することで、回答の信頼度を高める手法
Recursively Criticizes and Improves (RCI)	AIに自分の回答をチェックさせ「ここがダメだ」と反省して書き直させることで、質を高める手法
ReAct (Reason + Act)	「考える」だけでなく「検索する」などの行動を組み合わせ、正確な情報を元に答えを出す手法

Chain-of-Thought [[Wei\(Google\)+,2022](#)]

- 「答えを出すまでの考え方」を例示することで、「考える力」が劇的に向上
 - その後の手法 (ReAct や Self Consistency) も CoT の考え方がベース



- プロンプトに「ステップバイステップで」を追加すると計算などが正確になりやすい [[Kojima+,2022](#)]

引用: <https://arxiv.org/pdf/2201.11903.pdf>

ReAct (Reason + Act) [[Yao+,2022](https://arxiv.org/abs/2210.03629)]

- LLM に「考えさせてから行動させる」ことで、難しい質問への回答が正確になった
 - まず「何をすべきか(Act)」とその「理由(Reason)」をLLMが考える
 - その考えに沿って行動(例:検索)し、得られた結果から「次にすべきこと」と理由をまた考える

(1) Hotspot QA

Question: How many rooms are in the hotel that is home to the Cirque du Soleil show Mystere?

HotspotQA label: 2,664

Outdated ✗

(1a) Standard

Answer: 3,000 ✗

(1b) CoT (Reason Only)

Thought: Let's think step by step. The hotel that is home to the Cirque du Soleil show Mystere is Treasure Island. Treasure Island has 2,885 rooms, so the answer is 2,885.

Answer: 2,885 ✗

(1c) Act-Only

Act 1: Search[Cirque du Soleil]

Obs 1: Cirque du Soleil is a Canadian entertainment company...

Act 2: Lookup[Mystere]

Obs 2: No more results

Act 3: Search[Mystere (Cirque du Soleil show)]

Obs 3: Could not find...

Act 4: Search[Mystere (Cirque du Soleil)]

Obs 4: Mystère (my-steer) is one of six...

Act 5: Lookup[Treasure Island Hotel and Casino]

.....(end without answer) ✗

(1d) ReAct (Reason + Act)

Thought 1: I need to search Cirque du Soleil show Mystere, find the hotel it is in, then find the number of rooms in the hotel.

Act 1: Search[Cirque du Soleil show Mystere]

Obs 1: Could not find Cirque du Soleil show Mystere. Similar: ['Cirque du Soleil' ...

Thought 2: To find the hotel, I can search Mystere (Cirque du Soleil).

Act 2: Search[Mystere (Cirque du Soleil)]

Obs 2: Mystère (my-steer) is ... Held in a custom theatre at the Treasure Island Hotel and Casino ...

Thought 3: Mystere is held in a custom theatre at the Treasure Island Hotel and Casino. So I need to search Treasure Island Hotel and Casino next and find the number of rooms in the hotel.

Act 3: Search[Treasure Island Hotel and Casino]

Obs 3: Treasure Island Hotel and Casino ... with 2,884 rooms and 220 suites ...

Thought 4: Treasure Island Hotel and Casino has 2,884 rooms and 220 suites. So the answer is 3,104.

Act 4: Finish[3,104]

Up-to-date ✓

劇場を探す

ホテルを探す

部屋数を見
つける+合
算する

引用: <https://arxiv.org/abs/2210.03629>

- 小さなLLMに「時間をかけてじっくり考えさせる」と、大きなLLMと同等以上の性能を達成

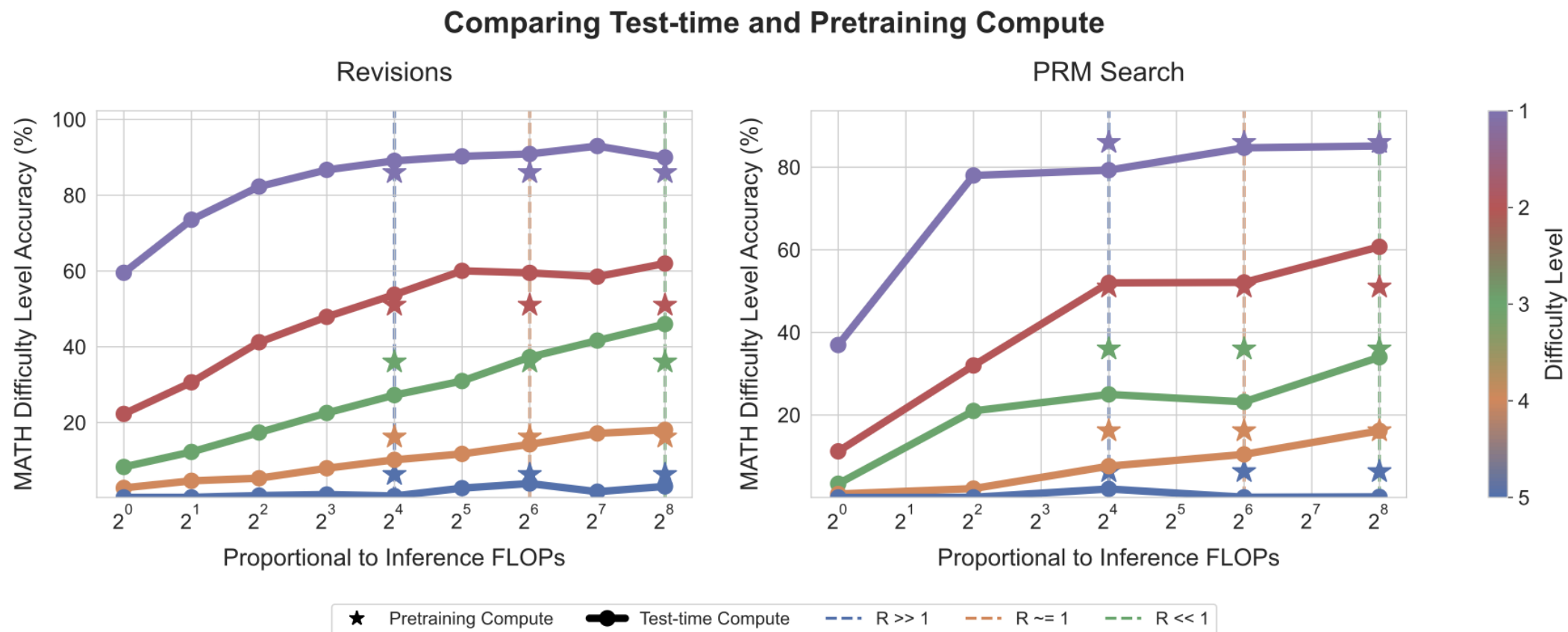
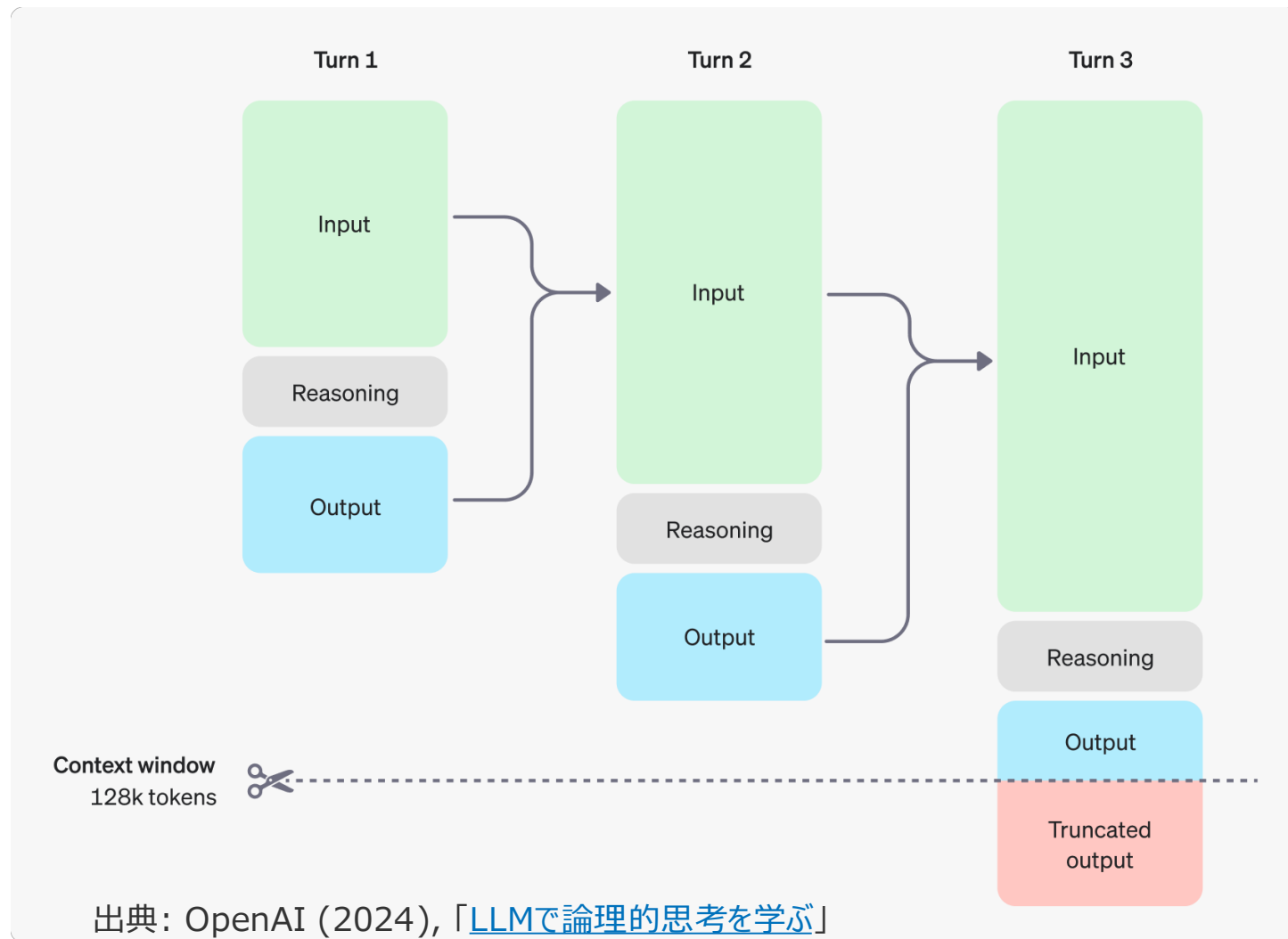


図: 推論時の計算予算を横軸に、難易度別に推論時スケーリング(●)と事前学習スケーリング(★)の優劣を示した図。簡単な問題では推論時スケーリング(紫や赤)が有利だが、難しい問題では事前学習の強化が有利(橙や青)になる。

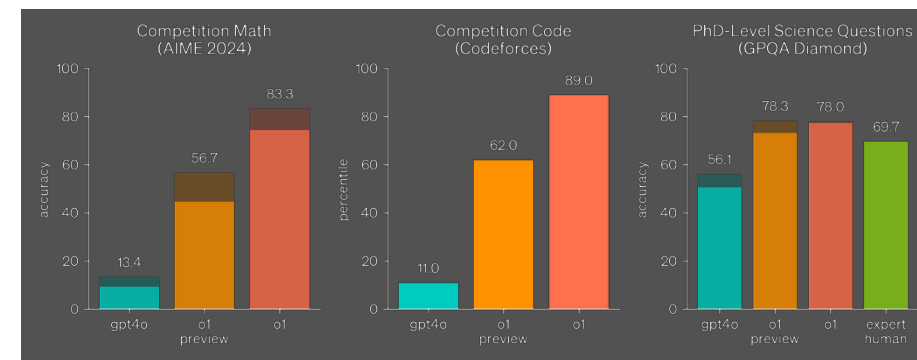
思考の深いモデル — OpenAI o1 (2024)

- 一つの質問に対して、人間のように「試行錯誤」して最適な答えを出す



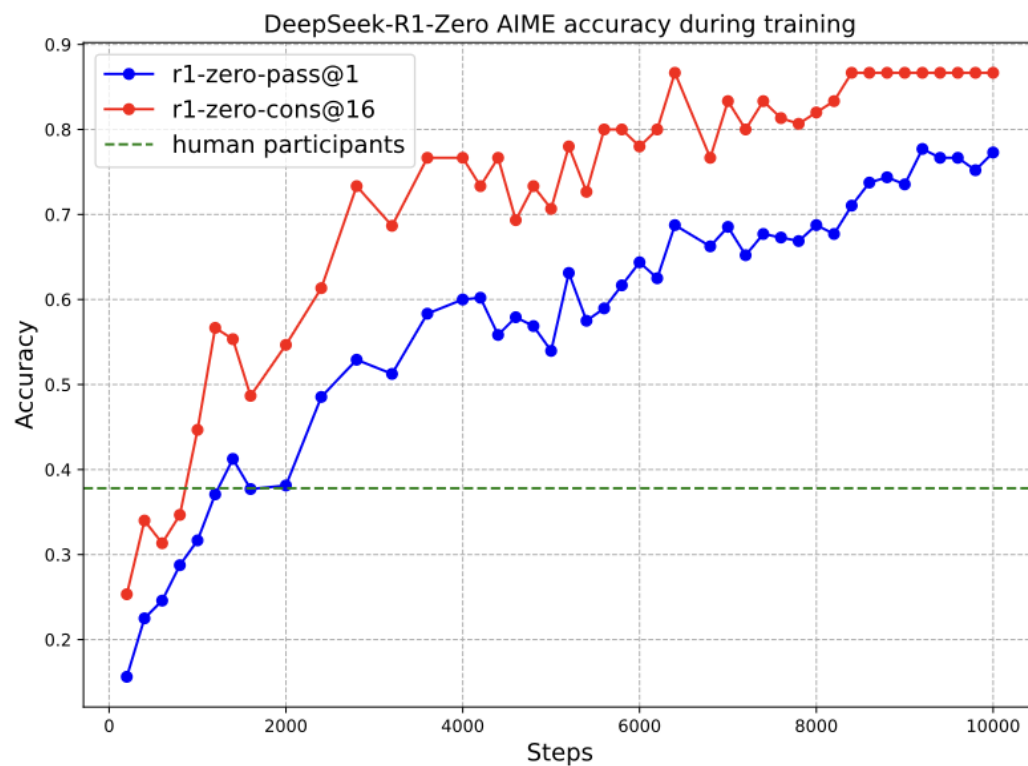
o1 の特徴:

- **人間のような思考:** いくつかの解決策を試し、間違いがないか自分でチェックして修正する
- **難問に強い:** 数学や科学など、高度な知識が必要な分野でこれまでの GPT-4o を圧倒した
- **使い分けが重要:** 「じっくり考える」 (=Reasoning) に特化した分、簡単な会話など全てのタスクで最適ではない

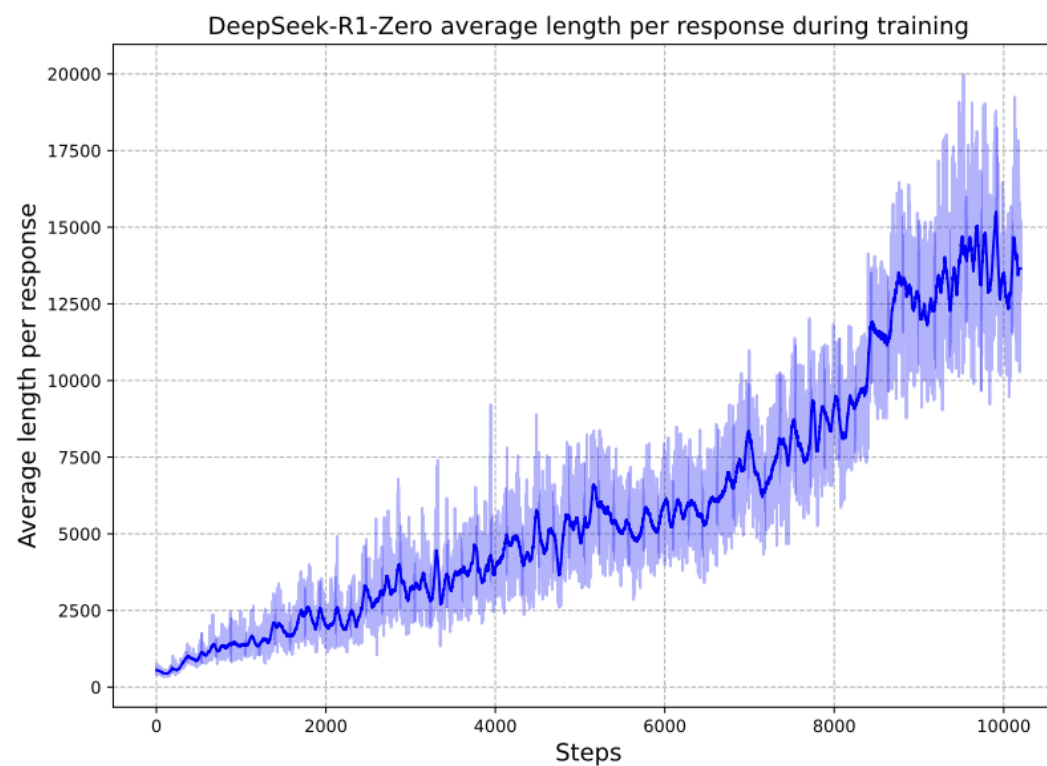


思考の深いモデル — DeepSeek-R1 (2025)

- DeepSeek-R1は、「正しい答えを出せたか」を学習し、OpenAI o1に近い性能
- 同じ質問への複数回答を比較してLLMを効率的に学習させる (GRPO)



左図: 複雑な問題(AIME)でも、人間のよう
に試行錯誤することで正解率が向上

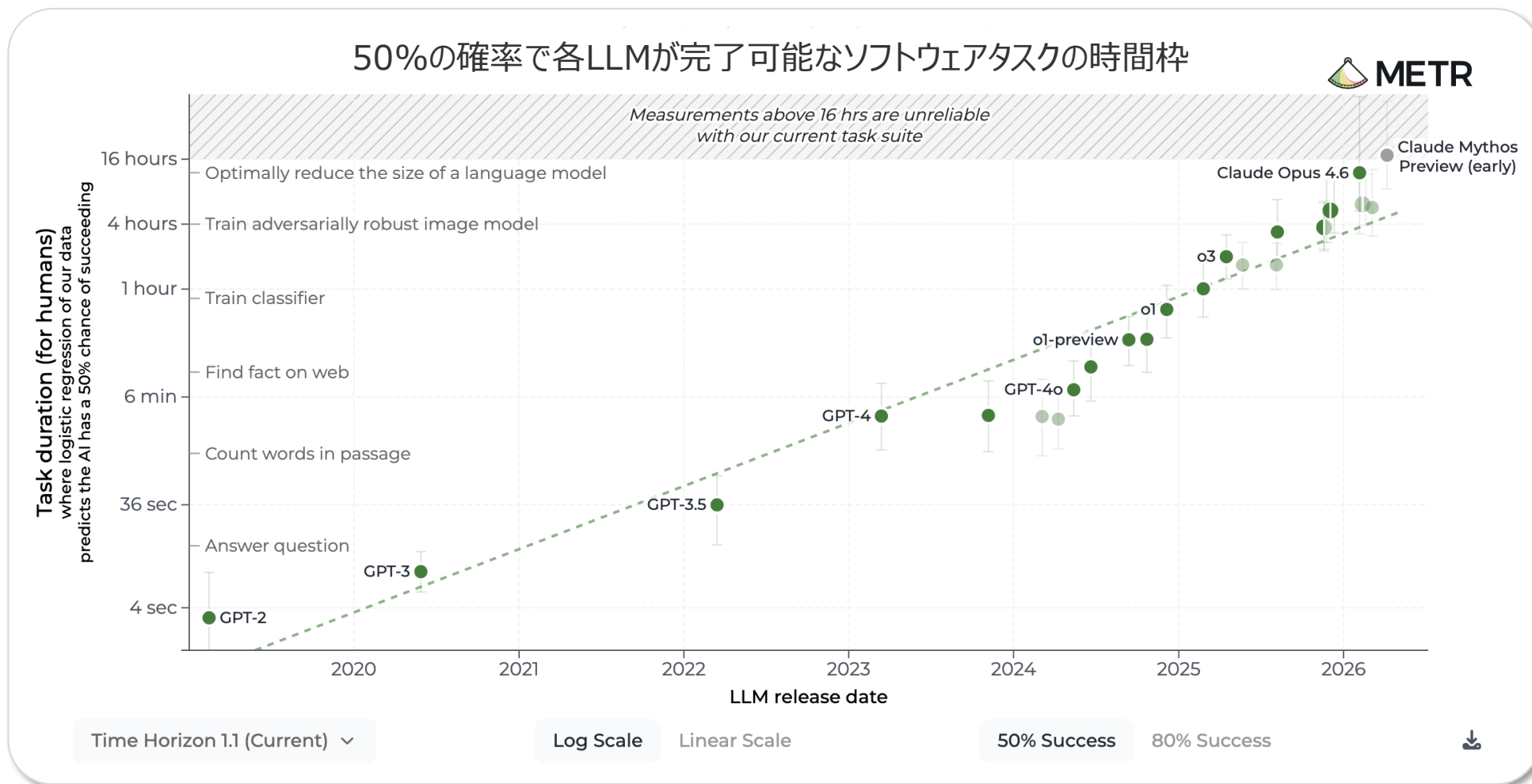


右図: RL訓練が進むと、教えられなくても自
発的に「長く、深く考える」ようになる

引用: <https://arxiv.org/abs/2501.12948>

自律的な実行能力 (深い思考+エージェント)

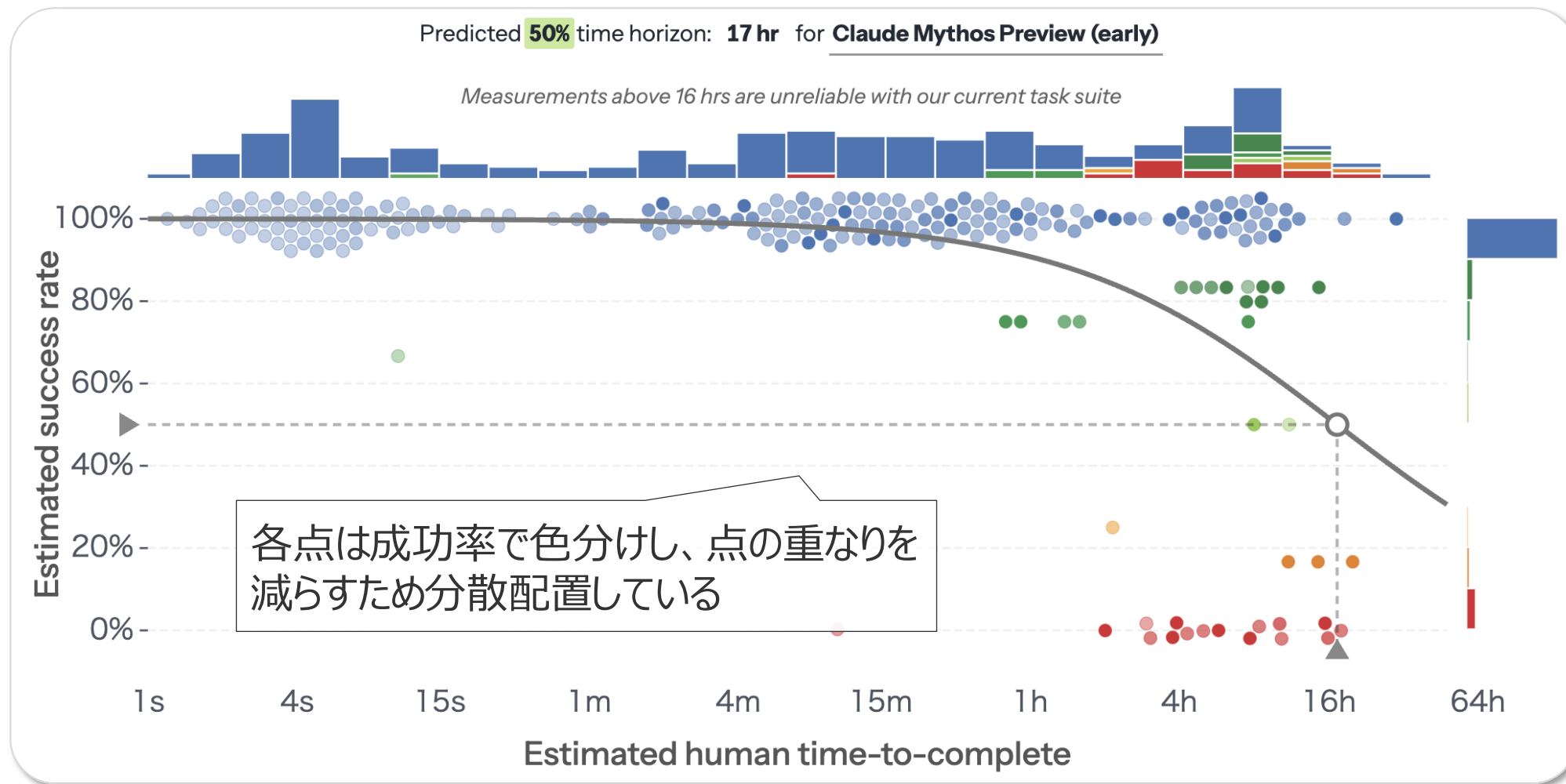
- Claude Opus 4.6 は人が**12時間**かかる作業を50%の確率で自力でやり遂げた



出典: [Task-Completion Time Horizons of Frontier AI Models](#) (METR)

自律的な実行能力 (深い思考+エージェント)

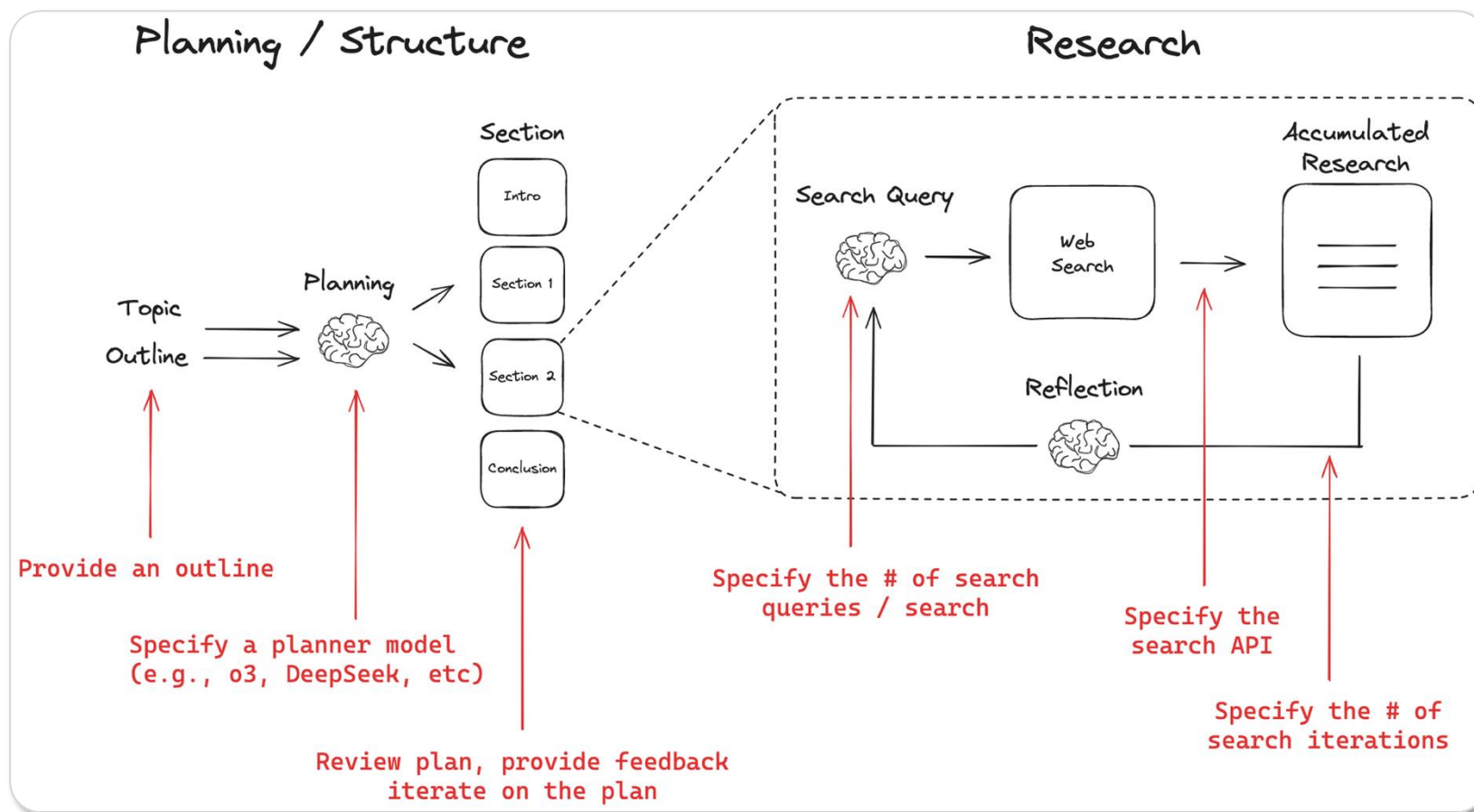
- Claude Mythos Preview は人が**17時間**かかる作業を50%の確率でやり遂げた



出典: [Task-Completion Time Horizons of Frontier AI Models](#) (METR)

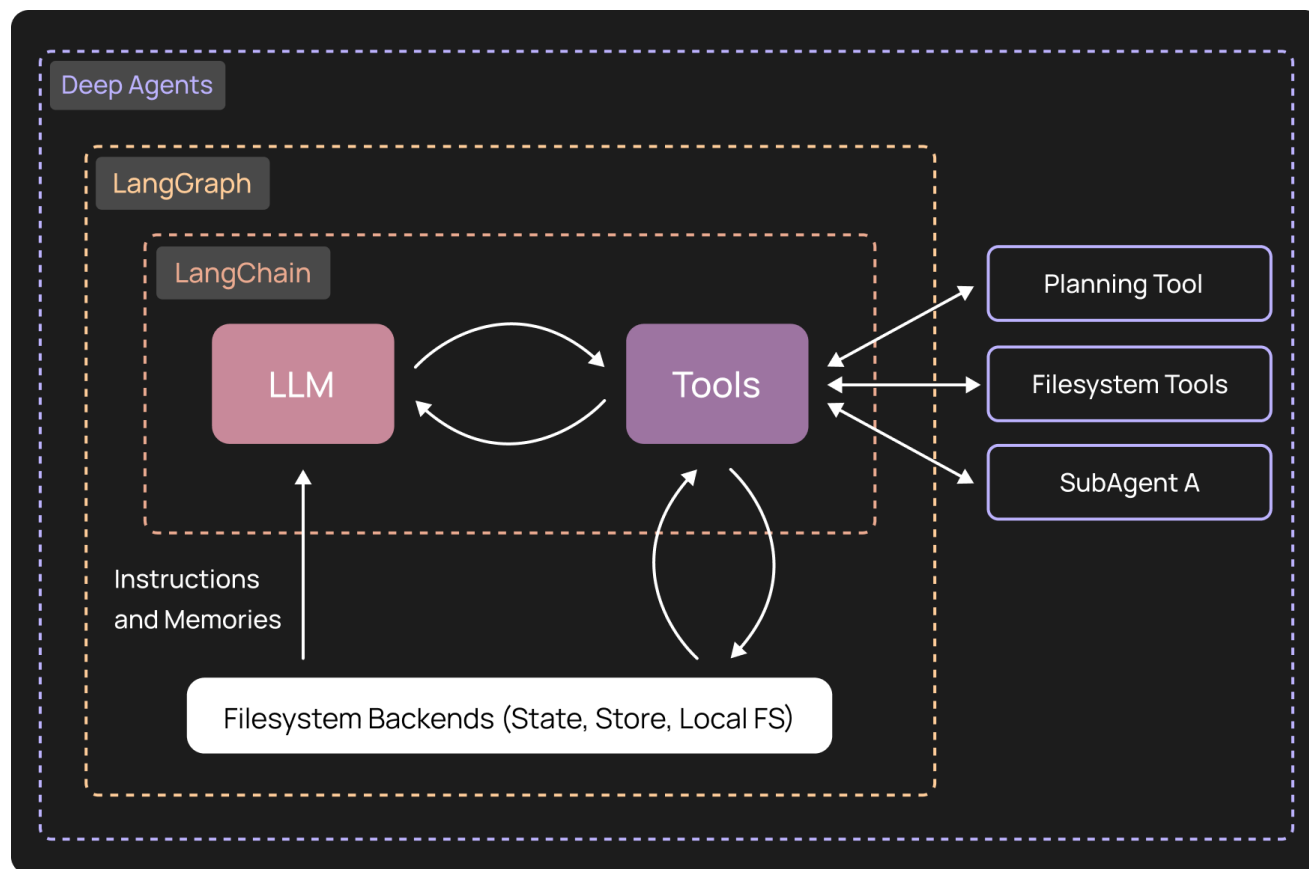
LLMエージェント — Open Deep Research

- OpenAI など Deep Research に対抗したオープンソース
 - LLMが作業をバラバラに分解し、計画・調査・実行を一つずつ確認しながら進める



出典: [Open Deep Research](#) (LangChain、動画)

- **エージェント = モデル (脳) + ハーネス (馬具)**
 - ハーネスとは、エージェントが迷わず仕事をするための「ルールと道具」



「ハーネス」に含まれる具体例:

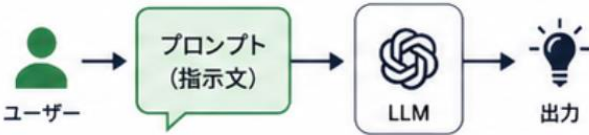
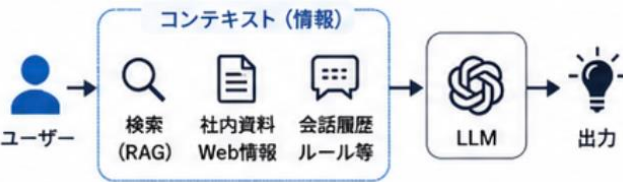
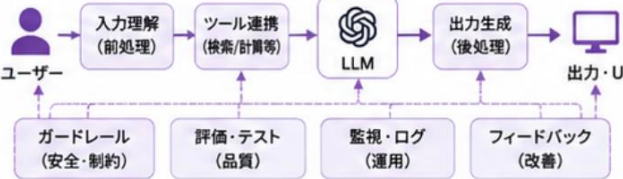
- 仕事の進め方を示す「指示書」
- 使える「道具」(ブラウザ、ファイル操作など)
- チーム体制 (サブエージェントへの役割分担)
- 仕事の「一貫性」を保つための自動チェック
- 作業の「記録」と「コスト・時間の測定」

※ これらがあるからこそ、エージェントは長時間一人で仕事を続けられる

出典: [Doubling down on Deep Agents](#) (LangChain)

プロンプト → コンテキスト → ハーネス

- Claude Code等のツールが用意した Harness 上に、自分たちの仕事に合わせたルールを適用や評価方法を整えることを **ハーネスエンジニアリング** と呼ぶ

	プロンプトエンジニアリング	コンテキストエンジニアリング	ハーネスエンジニアリング
目的	伝え方(指示)の最適化	渡す情報(文脈)の最適化	使い方(仕組み)の最適化
対象	指示文(Prompt)	LLMへ渡す入力情報全体	LLMを含むシステム全体
主な関心事	どう指示するか	何を渡すか	どう運用するか
主な活動	・役割設定 ・出力形式指定 ・Few-shot設計 ・制約条件設定	・RAG設計 ・情報収集・検索 ・履歴管理 ・コンテキスト構成	・ワークフロー設計 ・エージェント設計 ・ツール連携 ・評価・監視
典型例	「表形式で出力して」	「社内文書を検索して回答して」	「検索→要約→承認→実行のワークフロー」
イメージ図			

さらにその先へ、Agentic RL [[Zhang+,2025](#)]

- LLMおける強化学習の役割が、「人間好みに合わせる」から「自律的に考え・行動して課題解決する」へ変わる（例：OpenAI o3やおそらく最近のフロンティアモデル）

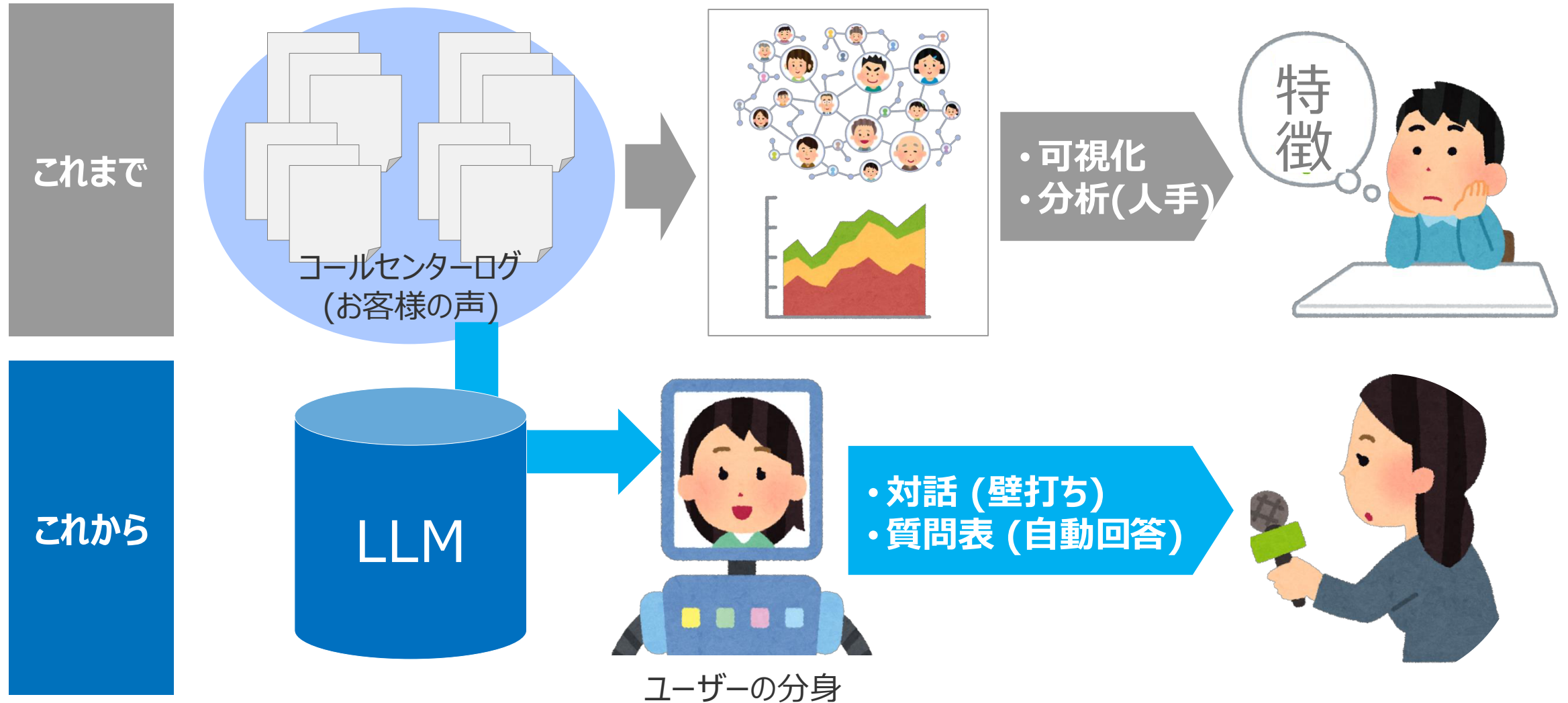
	従来のLLMの強化学習 (PBRFT)	Agentic RL
仕事の範囲 (状態/観測)	1つの質問に答えたらすぐ終了	自分で調べ、ツールを使い、完了するまで続く
できること (行動)	「文章を書く」だけ	文章作成 + 「Web検索やアプリ操作」
状況の変化 (遷移関数)	状況は変わらない	AIの行動によって状況がどんどん変わる
評価のされ方 (報酬)	最後の成果物に人間が「良い」「悪い」の点数を1回だけつける	最終ゴールだけでなく、途中の成果も評価 (例:「正しく検索できた」「エラーが出ずにツールを動かせた」など)
目指すゴール (目的関数)	その場その場で目の前の質問に人間が好む回答をすること	紆余曲折あっても、最終目的(長期的な)ゴールを達成すること

出典: [[Zhang+,2025](#)] The Landscape of Agentic Reinforcement Learning for LLMs: A Survey

テキストマイニングのこれから

テキストマイニングの未来予想

お客様の声の分析の例・・・これまでは大量のデータを人間が読み解いていたが、これからはユーザーの分身である、LLM との対話やヒアリングを通じて、本人も気づかない「本当の願い」や「お悩み」を簡単に見つけ出せる。



まとめ

- 大規模言語モデル(LLM)はどんなもの?
 - 生成AIは、我々の生活やビジネスに大きな影響を与え、利用範囲の拡大が進む
 - クローズドなLLM の開発と同様、ローカルLLM の開発競争も激しく、その差が縮まる
 - LLM によって、今後も多くの新しいサービスや応用が期待される
- 大規模言語モデルの成り立ち
 - 従来の言語モデルは文章を生成(次の単語を予測)していた
 - LLMは Transformerで文章生成する仕組みを超大規模化 →知識と読解力を得た
- テキストマイニングの未来予想
 - LLM によって、より手軽に誰でもテキストマイニングを実現できるようになる (予想)

スケジュール

day 1

- ・ 講義(後半) — 自然言語処理とLLM

day 2

- ・ 講義 — テキストマイニングの手順
- ・ 講義&演習 — データ理解
- ・ 講義&演習 — テキスト解析 (1)

day 3

- ・ 講義&演習 — テキスト解析 (2)
- ・ 講義&演習 — テキスト分析 (1)

day 4

- ・ 講義&演習 — テキスト分析 (2)

day 5

- ・ 演習 — テキスト分析 (発表会)
- ・ 講義&演習 — LLMを用いたテキスト分析
- ・ ラップアップ — Q&A

EXCEL

Google Colab

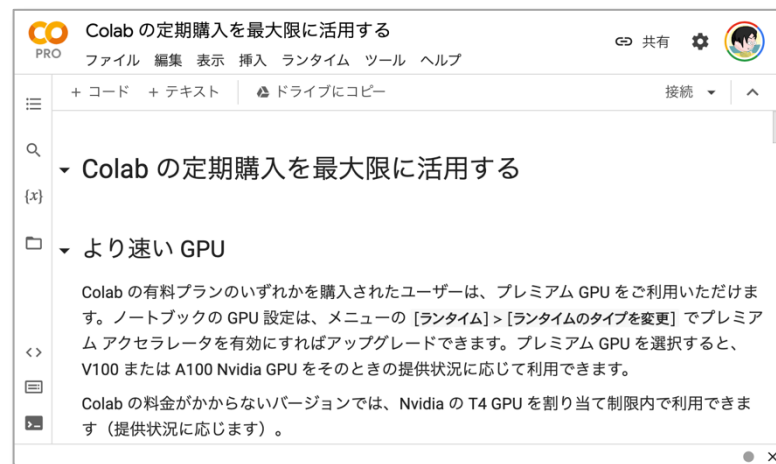
M365 Copilot Chat

【注意】次週までに, **Google アカウント**を作成しておいてください!

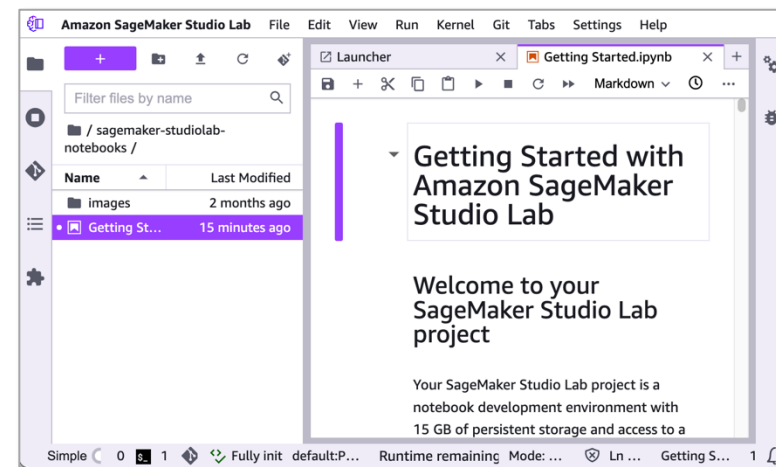
● 機械学習の教育・研究を目的とした研究用ツール



<https://colab.research.google.com>



<https://studiolab.sagemaker.aws/>



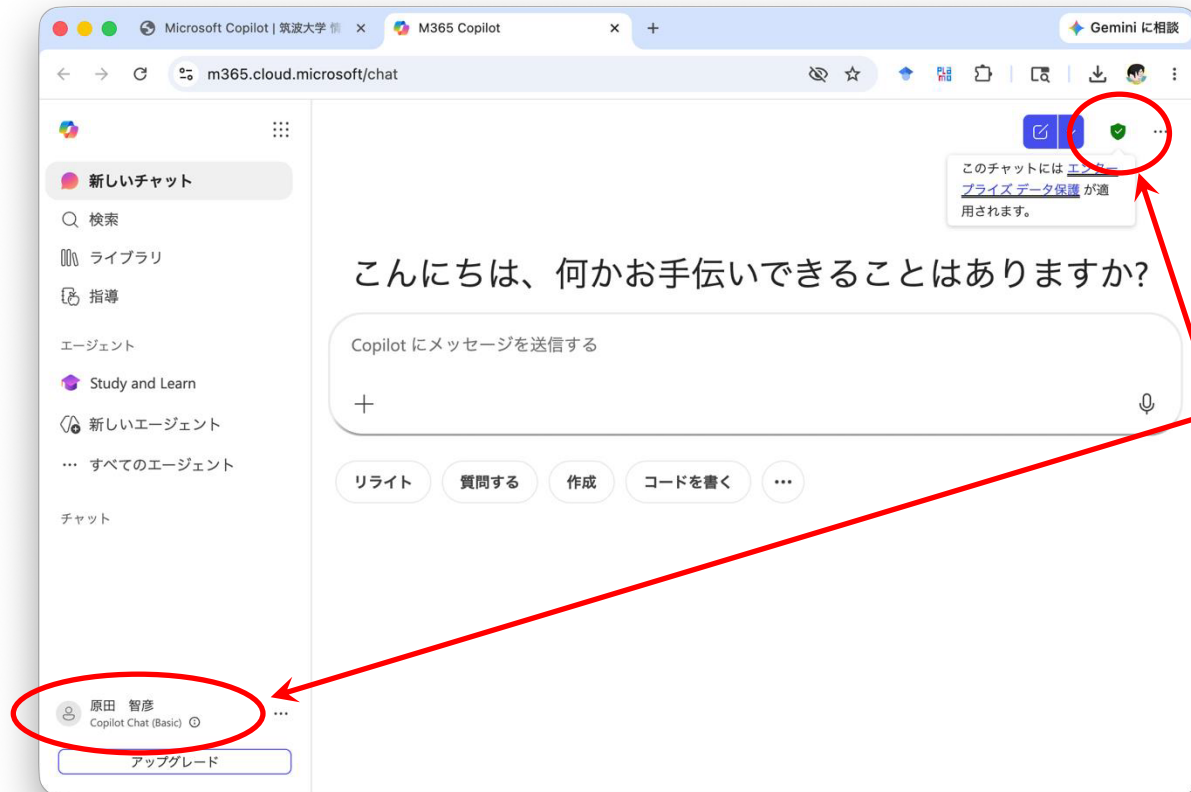
演習で使用



	Clab(無償版)	Studio Lab
GPU	T4(16GB)	T4(16GB)
最長 実行時間	12時間	CPU:12時間 GPU:4時間
メモリ	12GB	15GB
ディスク	CPU:100GB GPU:78GB	15GB (永続化)
ターミナル	△(限定的)	○
ランタイムの 保存と再開	×	○
費用	無償	無償
その他	Googleアカウントが必要	AWSアカウントは不要 (クレカ不要)

【注意】次週までに, **M365 Copilot Chat** のアカウントを作成しておいてください!

● Microsoft の「**Microsoft 365 Copilot Chat**」チャットが利用可能



Microsoft 365 Copilot Chat

利用対象者

@u または @un の本学のMicrosoft アカウントを持っている方
(離職・卒業・修了すると利用対象者ではなくなります)

利用手順

1. <https://m365.cloud.microsoft/> にアクセスします
2. 本学の @u または @un の Microsoft アカウントにてサインインしてください。
3. (既に、Microsoft 365 などにサインインしている場合でも、あらためてログイン操作が必要です)
4. Microsoft Copilot の画面が開きます。

右上に「**保護済み**」アイコンが表示され、カーソルをあわせると「このチャットにはエンタープライズデータ保護が適用されます」と表示されれば、大学の契約の下での「エンタープライズ データ保護を備えた Copilot チャット」を利用している状態です。

day 1 – レポート課題

- 以下を PDF ファイルで提出 してください
 - 社会人学生に役立つ、お薦めの生成AI(LLMやテキストマイニング以外も可)と使い方を教えてください
- ※ 何らかの事情で上記を提出できない場合、本日の講義の感想を文章で記述してください

レポート形式	提出先	期限
PDF	manaba	次回～18:20